

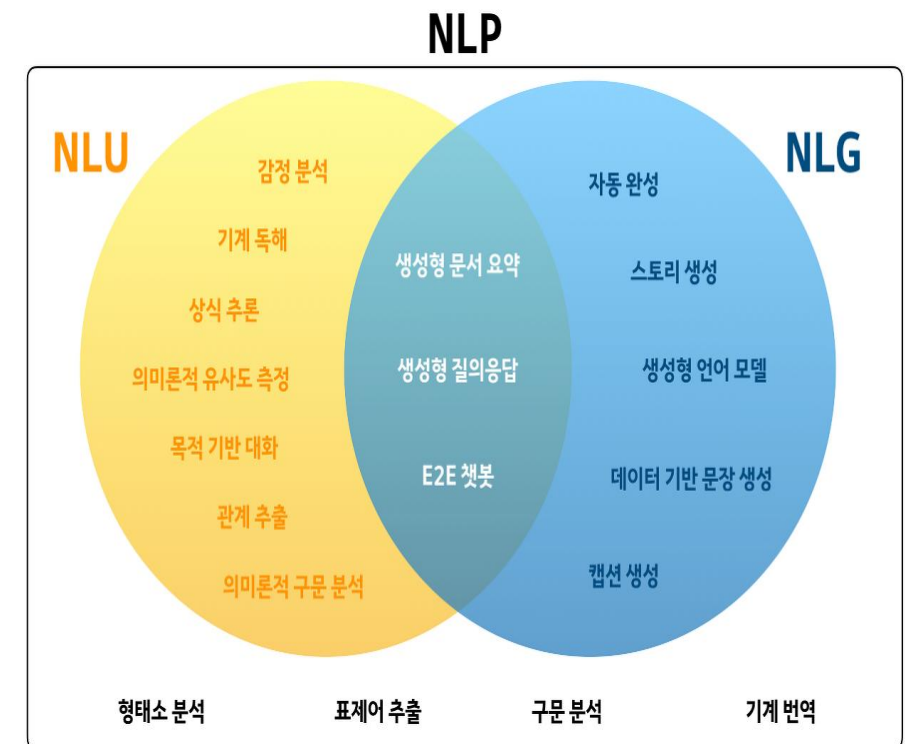
생성형 AI 개요

2026년 02월

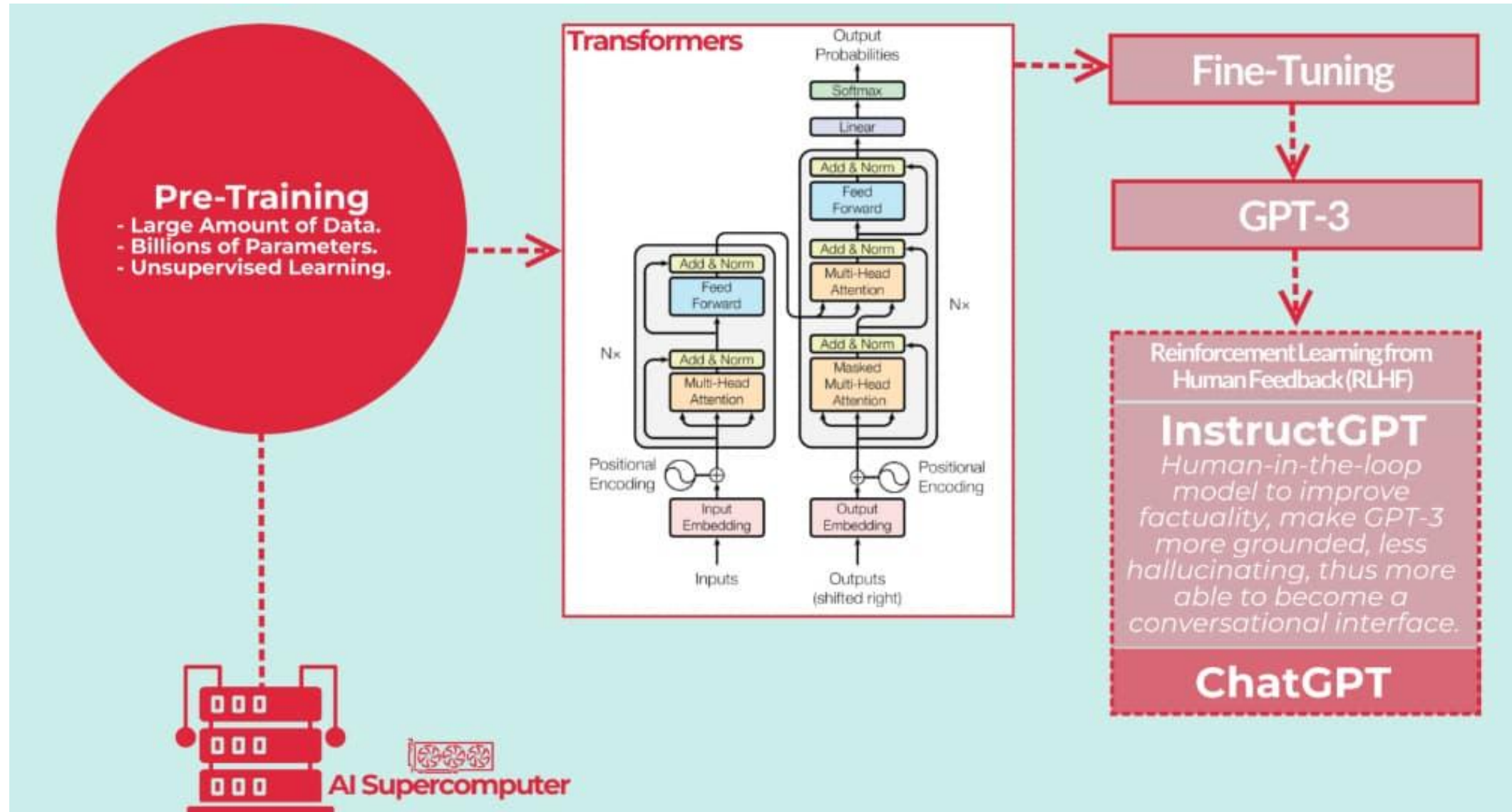
- 자연어 처리와 챗GPT
- 요약

자연어 처리와 ChatGPT

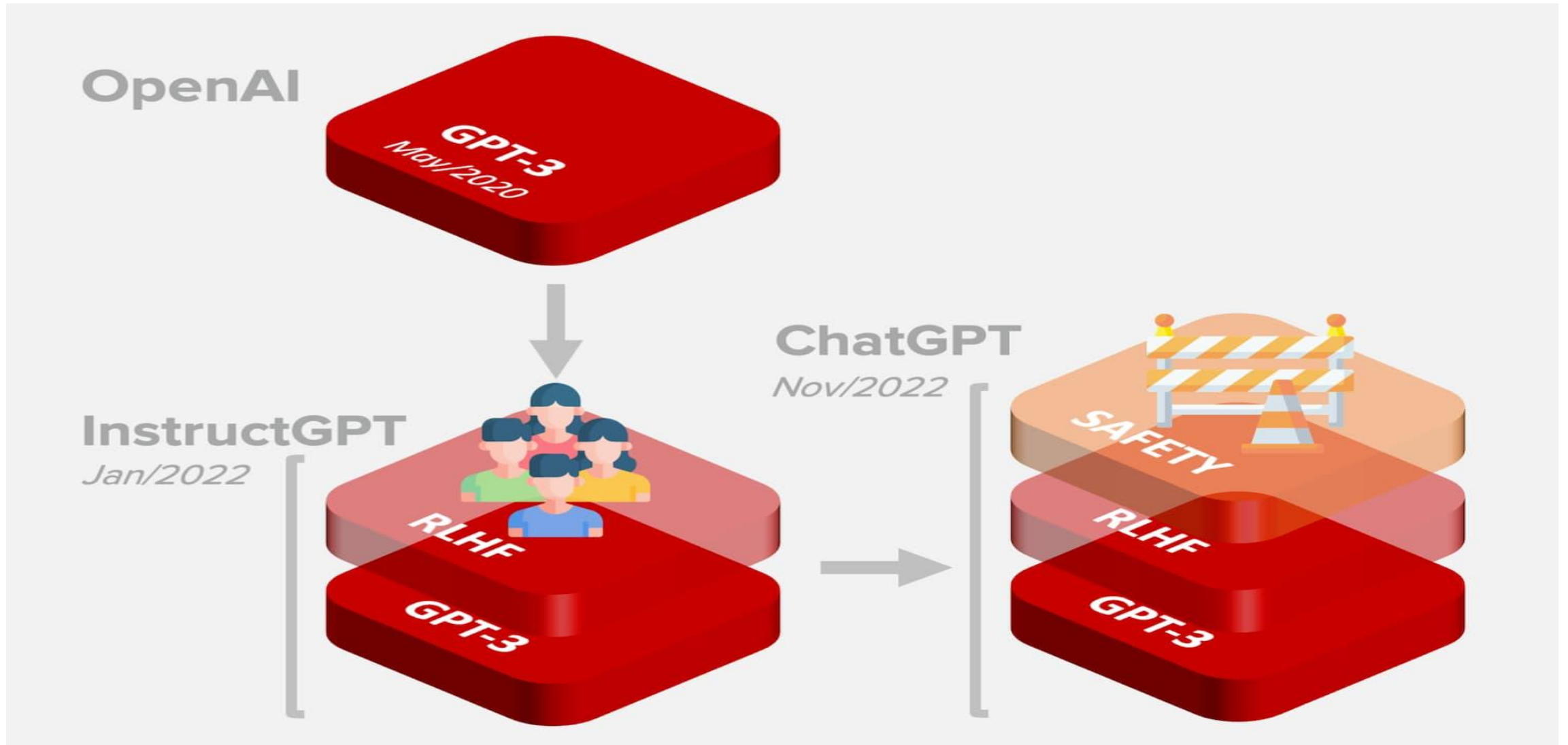
- 자연어(Natural Language) : 일상 생활에서 사용하는 언어
- 자연어 처리 : 자연어의 의미를 분석하여 컴퓨터가 처리할 수 있도록 하는 일
- 대표적인 NLP 연구 분야
 - ✓ 음성 인식, 문서 요약, 번역, 감성 분석
 - ✓ 텍스트 분류 : 스팸 분류, 뉴스 카테고리 분류
 - ✓ 질의 응답, 챗봇
- $NLP = NLU + NLG$

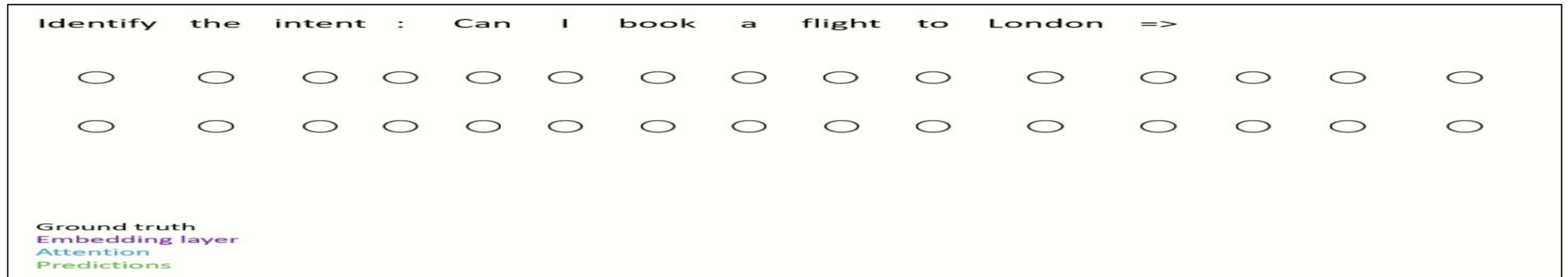
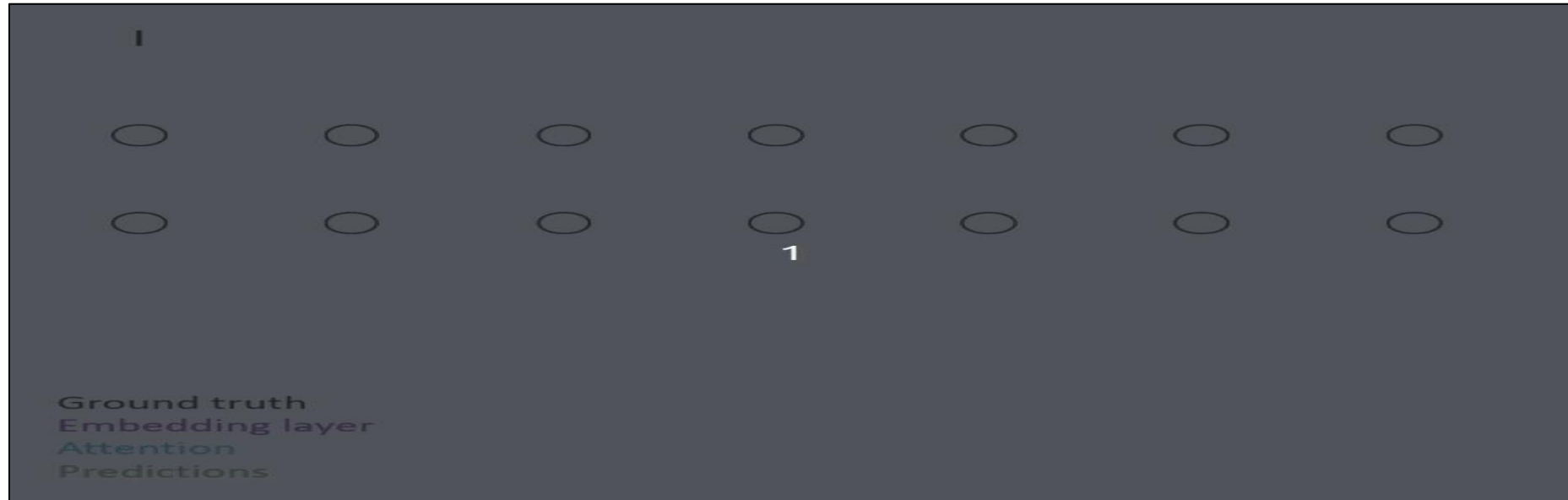


ChatGPT의 탄생



GPT, InstructGPT, ChatGPT

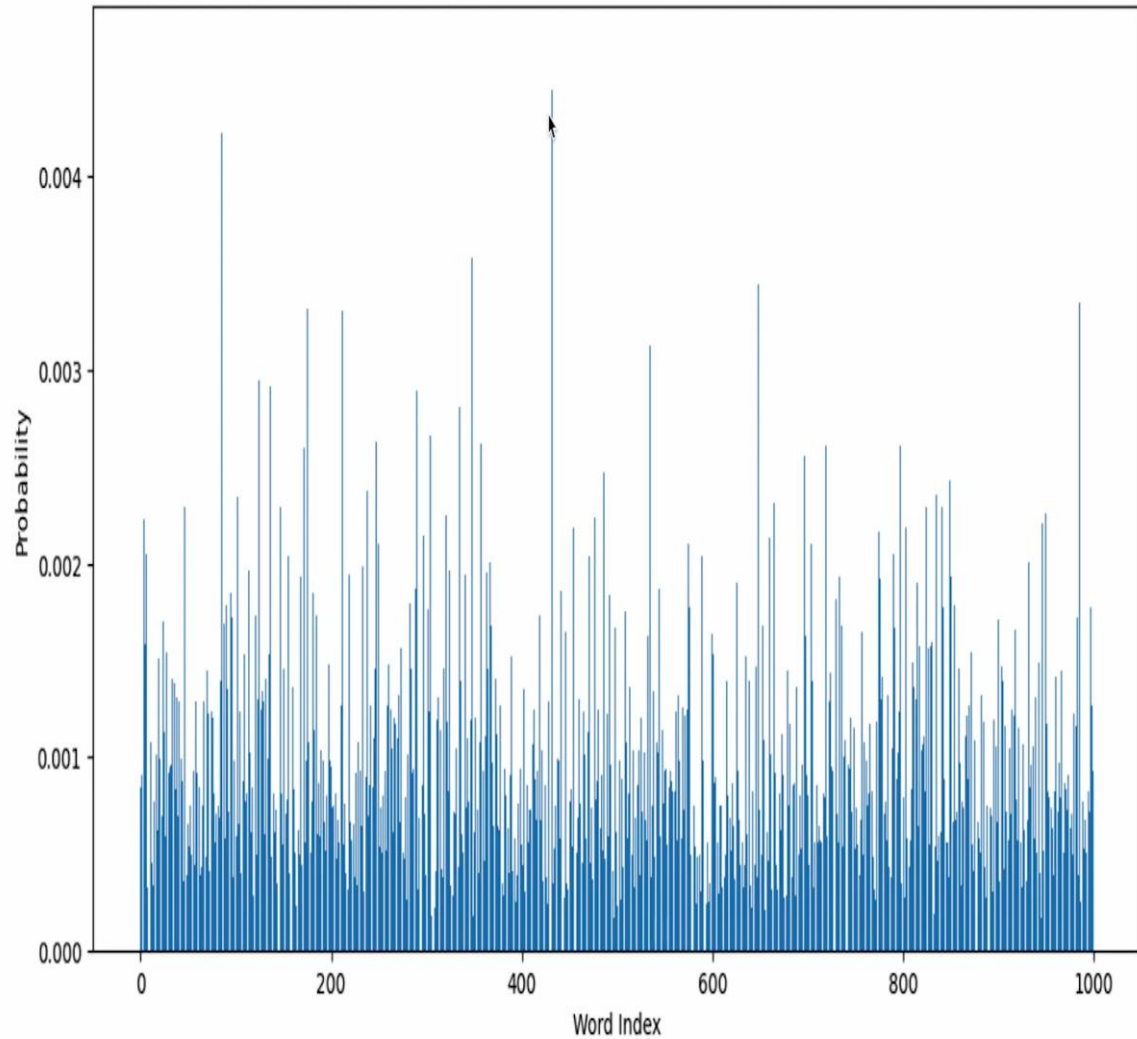




언어 생성의 원리 : 모델 훈련 전 vs 모델 훈련 후

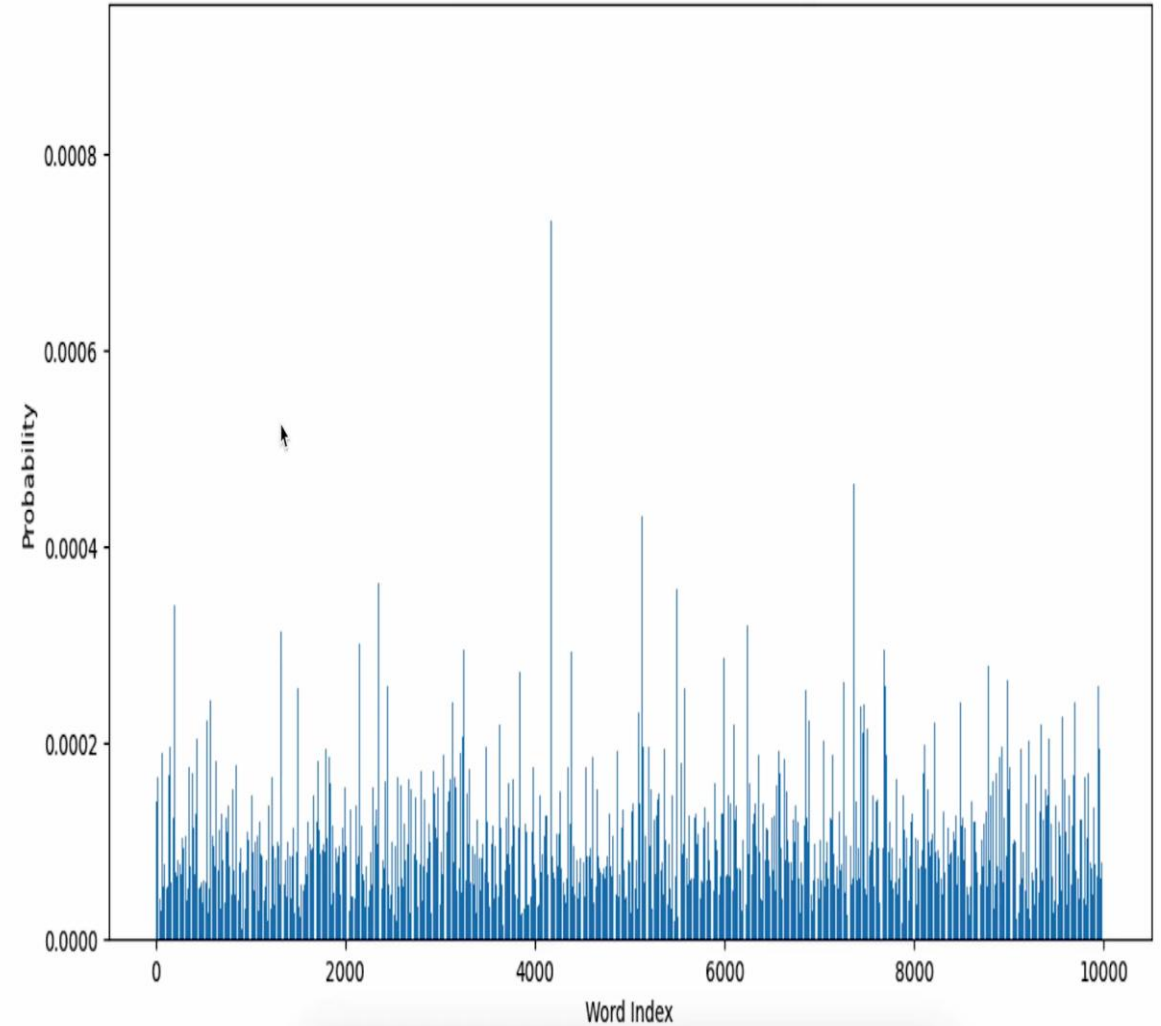


Output Distribution over Vocabulary



훈련 전

Output Distribution over Vocabulary



훈련 후

- 주어진 문장(질문)으로 다음 단어 예측
- 주어진 단어가 조합된 확률 기반
- 사실을 확인하는 장치 부족
➔ 환각(Hallucination) 발생
- 사실적 연관성 있는 질문을 해야~~

승우

백제 김유신 장군과 신라 계백 장군의 전투 파악해 줘 





백제 김유신 장군과 신라 계백 장군의 전투 (황산벌 전투)
배경

시대: 660년

상황: 백제 멸망 직전

주요 인물:

- 김유신: 신라의 뛰어난 장군, 나당 연합군 지휘
- 계백: 백제의 마지막 장군, 5천 결사대 지휘

전투 이유:

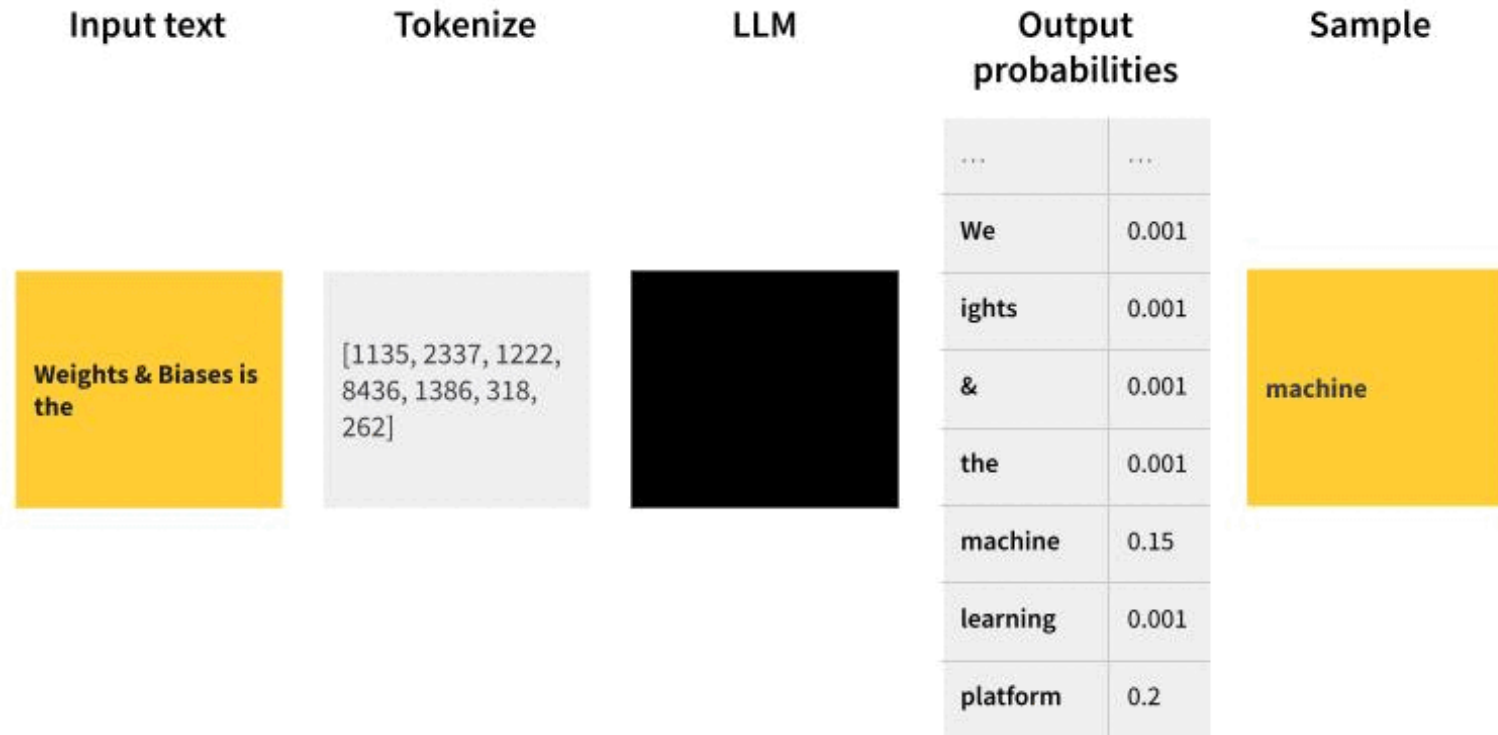
- 백제 멸망을 막기 위한 백제의 마지막 저항
- 신라의 삼국 통일을 위한 전략적 요충지 점령

전투 과정

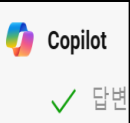
언어 생성의 원리 : Next Token Prediction



Predict the next token



- 주어진 문장(질문)으로 다음 단어 예측
- 주어진 단어가 조합된 확률 기반
- 사실을 확인하는 장치 부족
→ 환각(Hallucination) 발생
- NLU가 가능한 LLM
→ 문맥을 주면 읽는다
- 논리적 환각 현상



강고리

답변

김유신 장군은 고구려가 아닌 신라의 장군으로, 삼국 통일에 크게 기여한 인물입니다. 고구려의 장군 중에서 활약이 두드러진 인물은 연개소문이나 을지문덕 등이 있습니다. 그러나 김유신 장군의 활약상은 매우 중요하므로 신라의 김유신 장군에 대해 정리해 드리겠습니다.

김유신 장군의 주요 활약상

- 황산벌 전투 (660년)
 - 백제의 마지막 저항을 무너뜨린 전투.
 - 김유신은 신라군을 이끌고 백제의 계백 장군이 지휘하는 군대와 싸워 승리함으로써 백제를 멸

비어있는 5리터 물병과 15리터 물병이 있습니다.
5리터 물을 얻는 가장 단순한 방법은?

5리터 물을 얻는 가장 단순한 방법은 다음과 같습니다:

- 15리터 물병을 가득 채웁니다.
- 15리터 물병에서 5리터 물병에 물을 가득 채웁니다.
이로 인해 5리터 물병에는 5리터의 물이 있고, 15리터 물병에는 10리터의 물이 남습니다.
- 5리터 물병을 비웁니다.
- 15리터 물병에서 5리터 물병에 다시 물을 채웁니다.
이 과정에서 15리터 물병에는 정확히 5리터의 물이 남고, 5리터 물병에는 5리터의 물이 있게 됩니다.

이제 5리터 물병에 정확히 5리터의 물이 채워져 있습니다.

시킴으로써 고구려를 멸망시키

독립성을 지키는 데 중요한 역할

하는 국가로 발전할 수 있었음

논리력 테스트 : 문자 수 세기



strawberry 에는 알파벳 'r' 이 몇 개인가? 근거와 함께 답해 줘

Claude-3-Sonnet Poe

'strawberry'에는 알파벳 'r'이

근거:

1. 'strawberry'라는 단어를
2. 구체적으로는 'strawbe
3. 따라서 'strawberry'에



"Strawberry"라는 단어에는 알파벳 'r'이 **3개** 있습니다.

근거: 단어를 분석하면, 'strawberry'는 다음과 같이 나눌 수 있습니다:

- s
- t
- r
- a
- w
- b
- e
- r
- r
- y

따라서, 'r'이 3번 등장합니다.



strawberry 단어에는 알파벳 'r'이 2개 있습니다.

니다.

벳을 세어보면 총 10개의 알파벳으로 이루어져 있습니다.

와 네 번째 알파벳에 위치하고 있습니다.

는 알파벳 'r'이 2개 있습니다.

GPT-4o-Mini Poe

단어 "strawberry"에

근거:

- "strawberry"를
- s, t, r, a, w
- 여기서 'r'이 두

따라서, "strawberry

Poe

알파벳 'r' 이 **두 개** 있습니다.

다음과 같습니다:

라는 단어를 씁니다.

른쪽으로 글자를 보면서 'r' 을 찾습니다.

년 동그라미를 칩니다.

에는 동그라미 친 'r' 이 두 개 있습니다.

rry' 에는 'r' 이 두 개 있습니다.

나는 컵에 다이아몬드를 넣고 그 컵을 거꾸로 침대에 놓았다. 나중에 돌아와서 컵을 가져가서 냉장고에 넣었다. 다이아몬드는 어디에 있을까?



다이아몬드는 여전히 컵 안에 있습니다.

그 이유는 다음과 같습니다.

- 처음에 컵에 다이아몬드를 넣고 거꾸로 침대에 놓았습니다.
- 나중에 돌아와서 컵을 가져와 냉장고에 넣었습니다.
- 컵에서 다이아몬드를 꺼내지 않았다는 언급이 없으므로, 다이아몬드는 여전히 컵 안에 있다고 가정할 수 있습니다.

따라서 다이아몬드는 냉장고에 있는 컵 안에 있습니다.

나는 컵에 다이아몬드를 넣고 그 컵을 거꾸로 침대에 놓았다. 나중에 돌아와서 컵을 가져가서 냉장고에 넣었다. 다이아몬드는 어디에 있을까?



Copilot

다이아몬드는 컵을 냉장고에 넣었으니, 컵이나 냉장고에 있어야 합니다. 컵이나 냉장고가 열려 있으면 다이아몬드는 그곳에 있을 것입니다. 만약 컵이나 냉장고가 닫혔다면, 다이아몬드는 컵 안에 있어야 합니다. 따라서 컵을 열어 보면 다이아몬드를 찾을 수 있습니다.



다이아몬드는 **컵 안에** 있을 것입니다.

컵을 거꾸로 침대에 놓았을 때 다이아몬드는 컵 안에 머물렀습니다. 컵을 냉장고에 넣었을 때도 다이아몬드는 컵 안에 갇혀 있었기 때문에 컵 안에 그대로 남아있을 것입니다.

다른 가능성은 존재하지 않습니다. 다이아몬드가 컵에서 밖으로 나올 수 있는 방법은 없었기 때문입니다.

따라서 다이아몬드는 컵 안에 있는 것이 가장 확실합니다.

나는 컵에 다이아몬드를 넣고 그 컵을 거꾸로 침대에 놓았다. 나중에 돌아와서 컵을 가져가서 냉장고에 넣었다. 다이아몬드는 어디에 있을까?



다이아몬드는 침대 위에 있을 가능성이 큼니다. 다음은 그 이유에 대한 설명입니다:

1. 다이아몬드를 컵에 넣고 컵을 거꾸로 침대에 놓았다:

- 이 단계에서 다이아몬드는 컵 안에 있지만 컵이 거꾸로 놓였기 때문에 다이아몬드는 침대 표면에 닿아 있습니다.

2. 나중에 돌아와서 컵을 가져가서 냉장고에 넣었다:

- 컵을 거꾸로 놓은 상태에서 컵을 들어 올리면 다이아몬드는 컵 안에 있지 않고 침대 위에 남아 있게 됩니다.

따라서, 다이아몬드는 여전히 침대 위에 있을 것입니다. 냉장고에 넣은 컵에는 다이아몬드가 없을 것입니다.

나는 컵에 다이아몬드를 넣고 그 컵을 거꾸로 침대에 놓았다. 나중에 돌아와서 컵을 가져가서 냉장고에 넣었다. 다이아몬드는 어디에 있을까?

이 상황을 단계별로 분석해 보겠습니다:

1. 처음에 다이아몬드를 컵에 넣었습니다.
2. 그 컵을 거꾸로 침대에 놓았습니다.
3. 나중에 돌아와서 컵을 가져갔습니다.
4. 컵을 냉장고에 넣었습니다.

다이아몬드의 위치는 다음과 같을 것입니다:

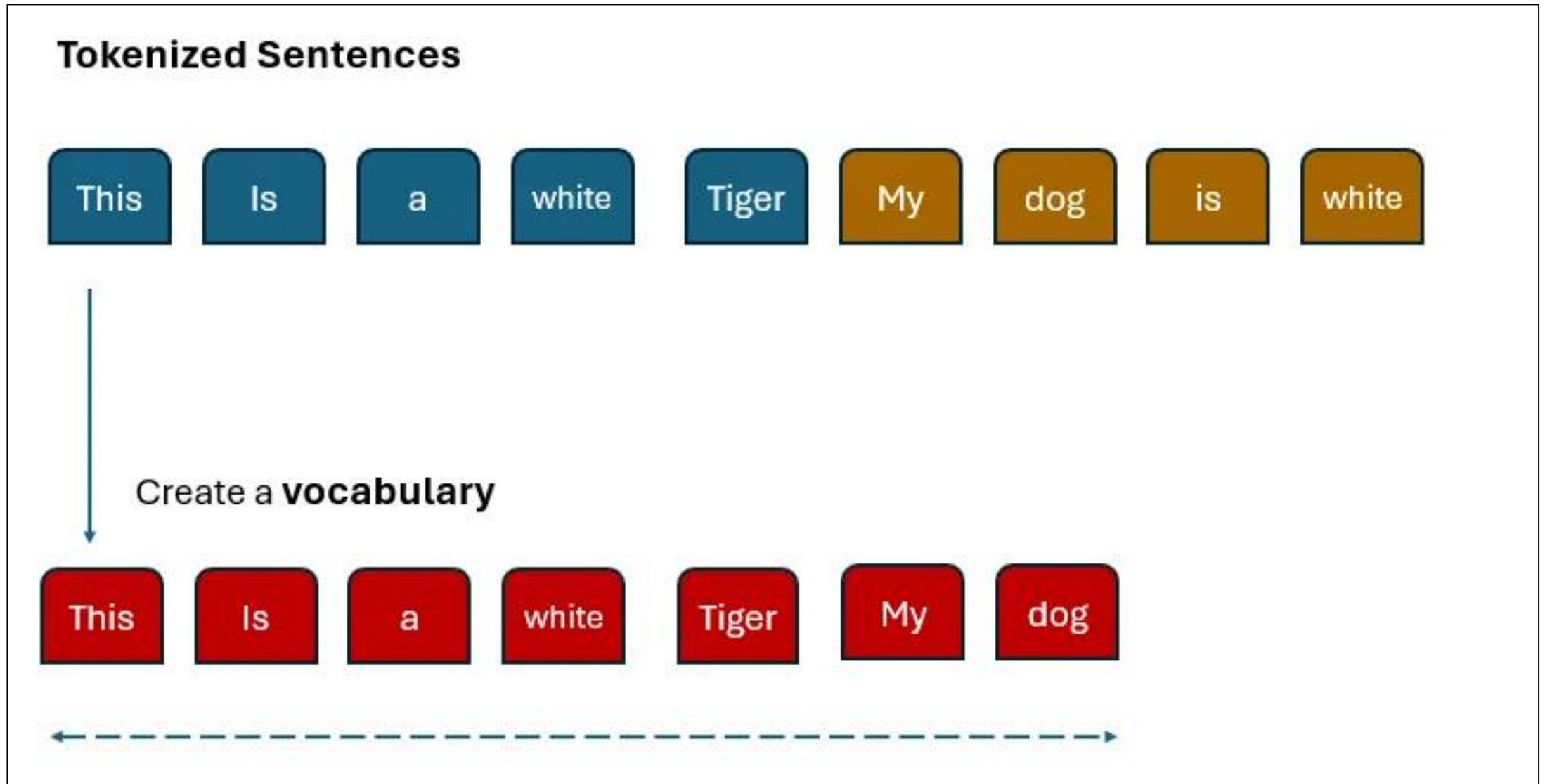
다이아몬드는 침대에 있을 가능성이 가장 높습니다.

이유:

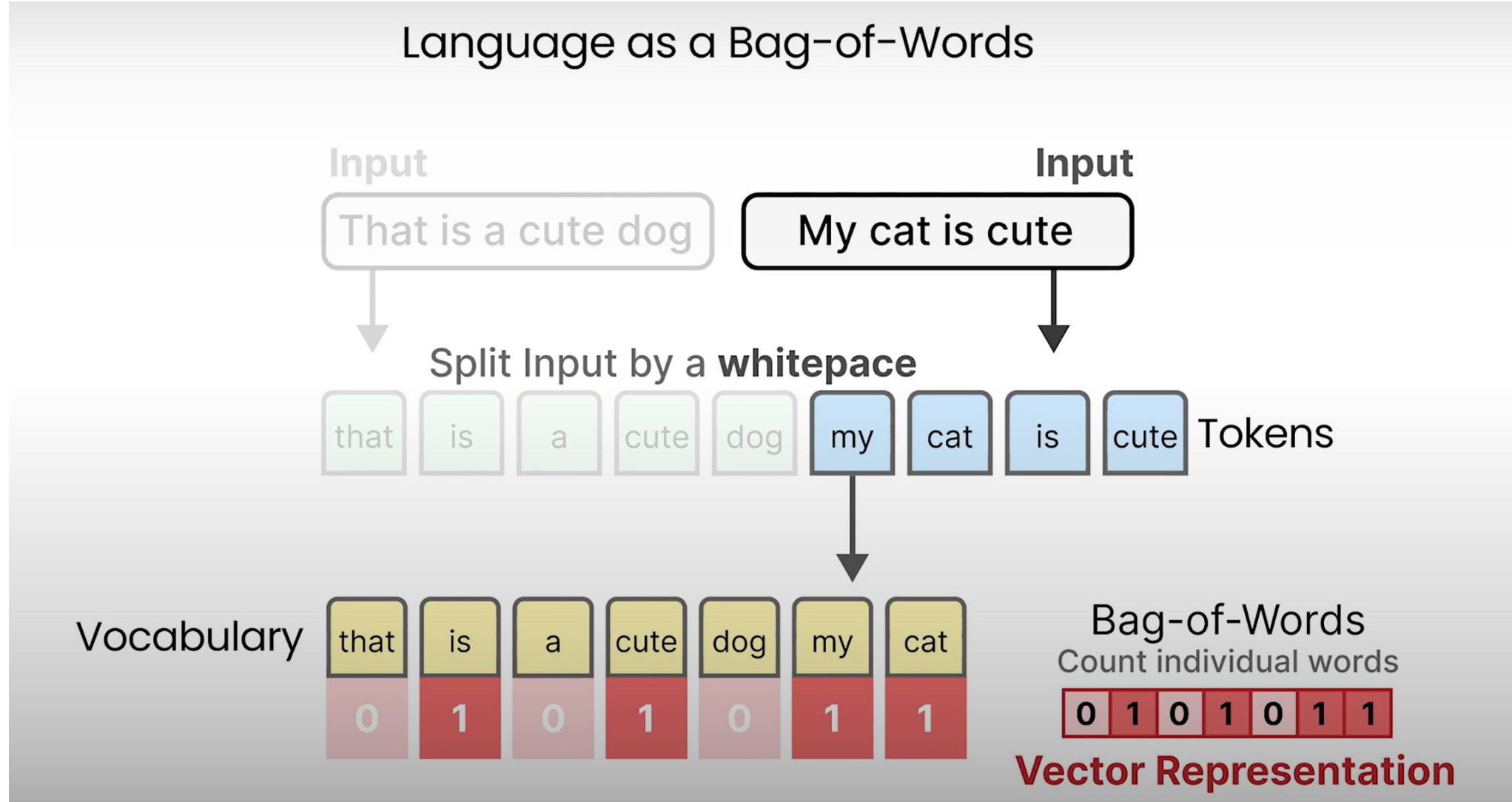
- 컵을 거꾸로 놓았을 때, 다이아몬드는 컵에서 떨어져 침대 위에 남았을 것입니다.
- 나중에 컵만 가져갔다면, 다이아몬드는 여전히 침대에 있을 것입니다.

이 추론이 맞는지 확인해 보시겠습니까? 혹시 다른 세부 사항이 있다면 알려주시면 더 정확한 답변을 드릴 수 있습니다.

Bag of Words : Token, Vocabulary



Bag of Words : 벡터 표현



LLM 모델 입출력 제약 : 토큰 제한 (Token Limit)



Model
Ada
Babbage
Curie
DaVinci
ChatGPT
GPT-4 8k context
GPT-4 32k context

모델	최대 토큰 수	설명
GPT-3.5	4,096	주로 간단한 추론 및 빠른 응답에 적합합니다.
GPT-3.5 Turbo	16,384	확장된 컨텍스트 창을 제공하며, 더 긴 텍스트 처리에 유리합니다.
GPT-4	8,192	복잡한 추론과 창의적인 작업에 적합하며, 약 6,000 단어를 처리할 수 있습니다.
GPT-4 (32k 버전)	32,768	약 50 페이지의 텍스트를 처리할 수 있는 확장된 컨텍스트 창을 제공합니다.
GPT-4 Turbo	128,000	매우 큰 컨텍스트를 처리할 수 있으며, 소설과 같은 긴 문서도 다룰 수 있습니다.
Claude 2	100,000	복잡한 논리적 작업에 적합하며, 약 60,000 단어를 처리할 수 있습니다.
Claude 3.5	100,000	향상된 처리 속도와 지능을 제공하며, 사용자 맞춤형 경험을 위한 메모리 기능을 포함합니다.
Llama 2	2,048	자연어 처리 및 대화 이해에 중점을 둔 모델입니다.
Llama 3.1	128,000	다양한 언어를 지원하며 최대 128K 토큰을 처리할 수 있습니다.

Token Limit	Words	Pages
4,096	1.5K	5
8,192	1.5K	5
32,768	3K	11
128,000	3K	11
100,000	6K	22
128,000	75K	272

출처 : [What Is the ChatGPT Token Limit and Can You Exceed It? \(makeuseof.com\)](https://makeuseof.com/what-is-the-chatgpt-token-limit-and-can-you-exceed-it/)

실습 : ChatGPT 토큰 (Link : <https://platform.openai.com/tokenizer>)



GPT-4

Chat

Model	Prompt	Completion	Model	Usage
8K context 32K context	GPT-3.5 & GPT-4 GPT-3 (Legacy) 나는 학교에 간다. Clear Show example Tokens 11 Characters 10 나는 에 다. TEXT TOKEN IDS	GPT-4o & GPT-4o mini GPT-3.5 & GPT-4 GPT-3 (Legacy) 나는 학교에 간다. Clear Show example Tokens 6 Characters 10 나는 학교에 간다. Text Token IDs		

[40, 467, 284, 1524, 13]

238, 220, 166, 108, 226, 46695, 97, 13]

(영어 단어 수) = 0.75*(토큰 수), (한글 글자수) = 0.33 *(토큰 수)
생성 콘텐츠 길이 제한, 가격 차이

엔트로픽 API 비용이 오픈AI보다 저렴하면서도 실제로는 더 비싼 이유

✎ 박찬 기자 ⌚ 입력 2025.05.03 21:20

엔트로픽의 '클로드'가 오픈AI 모델보다 API 비용이 저렴한 것으로 보이지만, 구조적인 문제로 인해 실제로는 더 비싸다는 지적이 나왔다.

벤처비트는 1일(현지시간) 공개된 API 가격이 저렴한 엔트로픽의 모델이 실제로는 오픈AI보다 20~30% 더 비쌀 수 있다고 보도했다.

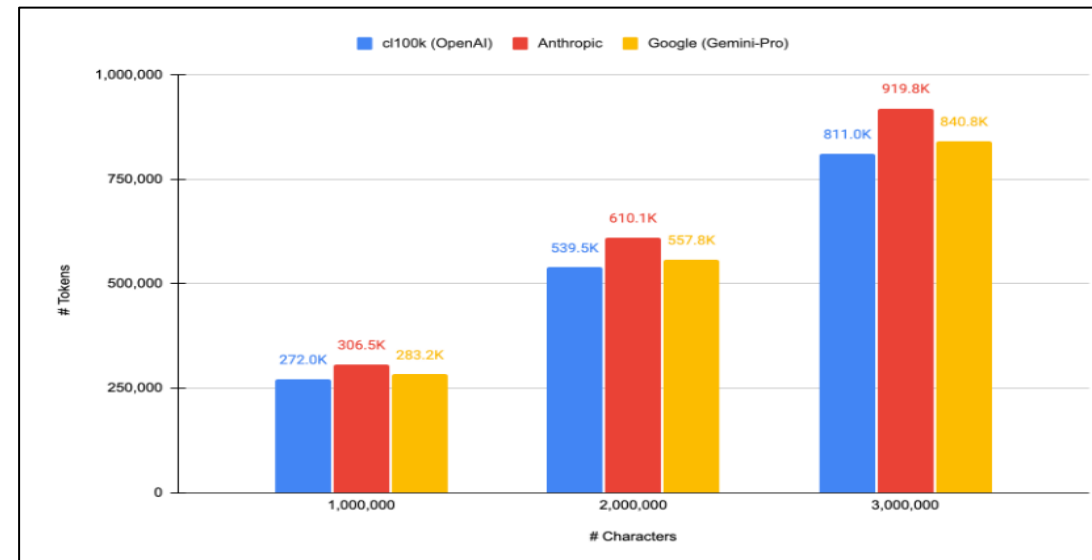
공개된 API 가격표에 따르면, 엔트로픽의 '클로드 3.5 소네트'와 오픈AI의 'GPT-4o'는 가격이 비슷한 것으로 보인다. 출력 토큰에 대한 비용은 같으며, 입력 토큰은 클로드 3.5 소네트가 40% 더 저렴하다.

그러나 프롬프트를 실제 입력한 실험 결과, GPT-4o가 전체 비용 면에서 클로드 3.5 소네트보다 훨씬 더 저렴한 것으로 나타났다.

이는 두 모델의 토큰화 방식 차이에 따른 것이다. 엔트로픽의 토큰나이저(tokenizer)는 같은 입력에 대해 오픈AI의 토큰나이저보다 더 많은 토큰으로 분할하는 경향이 있다. 즉, 같은 프롬프트라고 해도 클로드는 더 많은 토큰을 생성하게 돼, 입력 토큰 비용이 더 낮아도 전체 비용은 더 커질 수 있다는 것이다.

또 커뮤니티의 실험에 따르면, 구글의 '제미니'는 오픈AI보다는 조금 더 많은 토큰을 생성하지만, 클로드에는 못 미치는 것으로 나타났다.

더 많은 토큰을 만드는 오버헤드 문제는 콘텐츠 유형에 따라 달라진다. 영어로 된 문장에서는 클로드 3.5 소네트의 토큰나이저가 GPT-4o보다 16% 더 많은 토큰을 생성한다. 더 구조적이거나 기술적인 콘텐츠에서는 더 증가한다. 수학 방정식에서는 오버헤드가 21%, 파이썬 코드에서는 30%에 달한다.



이런 차이는 엔트로픽의 토큰나이저가 코드와 수학적 문서에서 자주 등장하는 패턴과 기호를 더 작은 토큰으로 분할하기 때문에 발생한다. 반면, 자연어 콘텐츠는 토큰화 오버헤드가 적다.

이런 문제는 비용뿐만 아니라, 컨텍스트 창 활용에도 영향을 미친다. 엔트로픽의 모델은 20만개의 토큰을 처리할 수 있는 큰 컨텍스트 창을 제공하지만, 추가 토큰으로 인해 실제 사용할 수 있는 공간은 줄어들 수 있다. 즉, 공개된 컨텍스트 창 크기와 실제 사용할 수 있는 컨텍스트 창 크기 사이에 차이가 있을 수 있다는 분석이다.

토큰화의 숨겨진 비용은 당연히 개발자와 기업에 영향을 미칠 수 있다. 실제 사용에서는 오픈AI 모델이 더 경제적일 수 있기 때문이다.

또 단순한 토큰당 비용만을 고려할 것이 아니라, 토큰화 방식이 전체 성능과 비용 효율성에 미치는 영향을 함께 고려해야 한다는 것을 의미한다.

실습 : ChatGPT 토큰 (Link : <https://platform.openai.com/tokenizer>)



GPT-3.5 & GPT-4

GPT-3 (Legacy)

ha|

Clear

Show example

Tokens

1

Characters

2

ha

Text

Token IDs

GPT-3.5 & GPT-4

GPT-3 (Legacy)

hap

Clear

Show example

Tokens

1

Characters

3

hap

Text

Token IDs

GPT-3.5 & GPT-4

GPT-3 (Legacy)

happ|

Clear

Show example

Tokens

2

Characters

4

happ

Text

Token IDs

GPT-3.5 & GPT-4

GPT-3 (Legacy)

happy

Clear

Show example

Tokens

1

Characters

5

happy

Text

Token IDs

GPT-3.5 & GPT-4

GPT-3 (Legacy)

happily|

Clear

Show example

Tokens

3

Characters

7

happily

Text

Token IDs

AI 인식의 구조적 차이



```
prompt == How many r's in the word strawberry?  
That's an easy one! There are 2 R's in the word "strawberry".None
```

```
prompt == How many r's in the word s t r a w b e r r y?  
There are 3 R's in the word "strawberry".None
```

How many r's in the word strawberry?

How many r's in the word s t r a w b e r r y?

3.11 과 3.9 중 어느 숫자가 커?

3.11이 3.9보다 큼니다. 소수점 첫 번째 자리까지 같지만, 두 번째 자리에서 1이 9보다 크므로 3.11이 더 큰 숫자입니다.



GPT-4o & GPT-4o mini

GPT-3.5 & GPT-4

GPT-3 (Legacy)

3.11 과 3.9 중 어느 숫자가 커?

Clear

Show example

Tokens

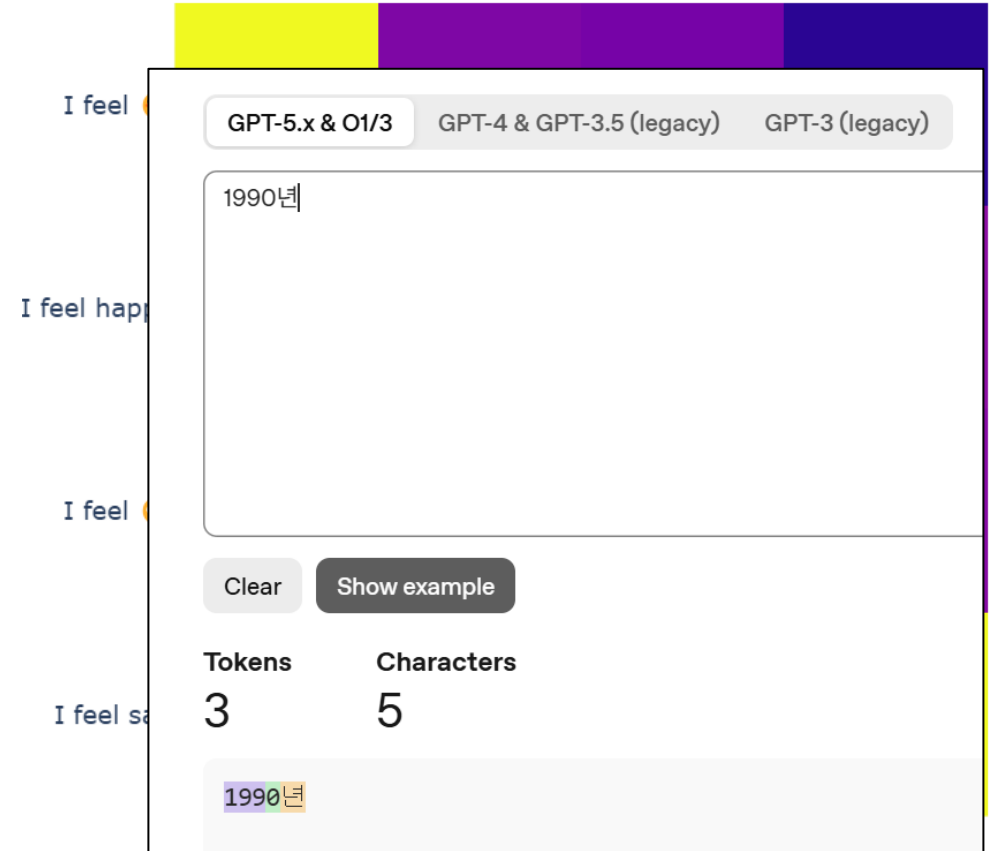
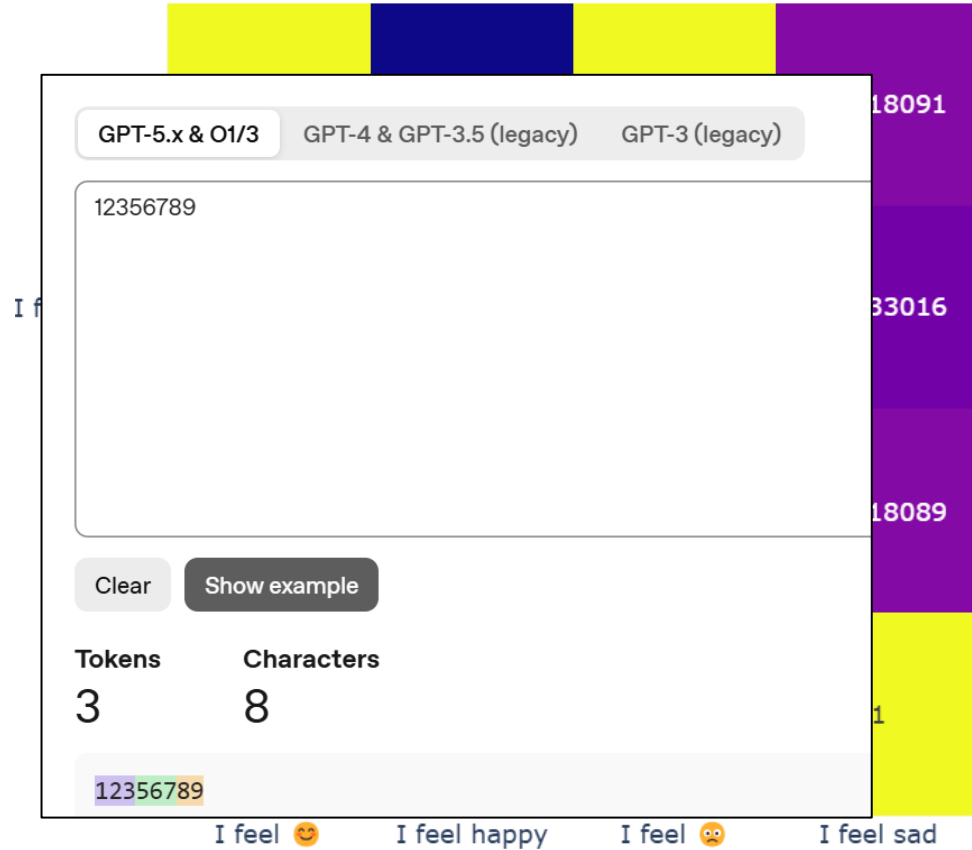
14

Characters

22

3.11 과 3.9 중 어느 숫자가 커?

AI 인식의 구조적 차이



- Tokenizer 의 영향 : Unknown, 숫자, 날짜 , 오타

실습 링크 : <https://colab.research.google.com/drive/1q1y5g5QHmb2O6FYYjc3OEekMSyHSQo7f?usp=sharing>

"AI 게임체인저"...LG, 독자 개발 'K-엑사원' 공개

등록 2026.01.11 10:00:00 | 수정 2026.01.11 10:14:24



LG AI연구원은 AI의 언어 능력 향상에 중요한 역할을 하는 토크나이저(Tokenizer)도 고도화했다.

토크나이저는 AI가 이해하는 단위인 토큰으로 문장을 쪼개는 기술이다.

LG AI연구원은 학습 어휘를 15만 개로 확장하고, 자주 쓰는 단어 조합은 하나로 묶는 방식을 적용하는 등 토크나이저 고도화로 'K-엑사원'이 기존 모델 대비 1.3배 더 긴 문서를 기억하고 처리할 수 있게 했다.

또 LG AI연구원은 하나의 토큰(Token)을 처리하면서 다음 토큰을 예측할 수 있는 멀티 토큰 예측(MTP) 영역을 설계해 추론 속도를 기존 모델 대비 150% 높였다.

LG AI연구원 관계자는 "'K-엑사원'은 효율은 높이고 비용은 낮추는 모델 설계를 통해 고가의 인프라가 아닌 A100급의 GPU 환경에서도 구동할 수 있다"라며, "인프라 자원이 부족한 기업들도 프런티어급 AI 모델을 도입해 활용할 수 있도록 함으로써 국내 AI 생태계 저변을 넓히고자 한다"고 밝혔다.

SuperBPE: Space Travel for Language Models

*Alisa Liu^{♡♠} *Jonathan Hayase[♡]

Valentin Hofmann^{◇♡} Sewoong Oh[♡] Noah A. Smith^{♡◇} Yejin Choi[♠]

[♡]University of Washington [♠]NVIDIA [◇]Allen Institute for AI

BPE: By the way, I am a fan of the Milky Way.

SuperBPE: By the way, I am a fan of the Milky Way.

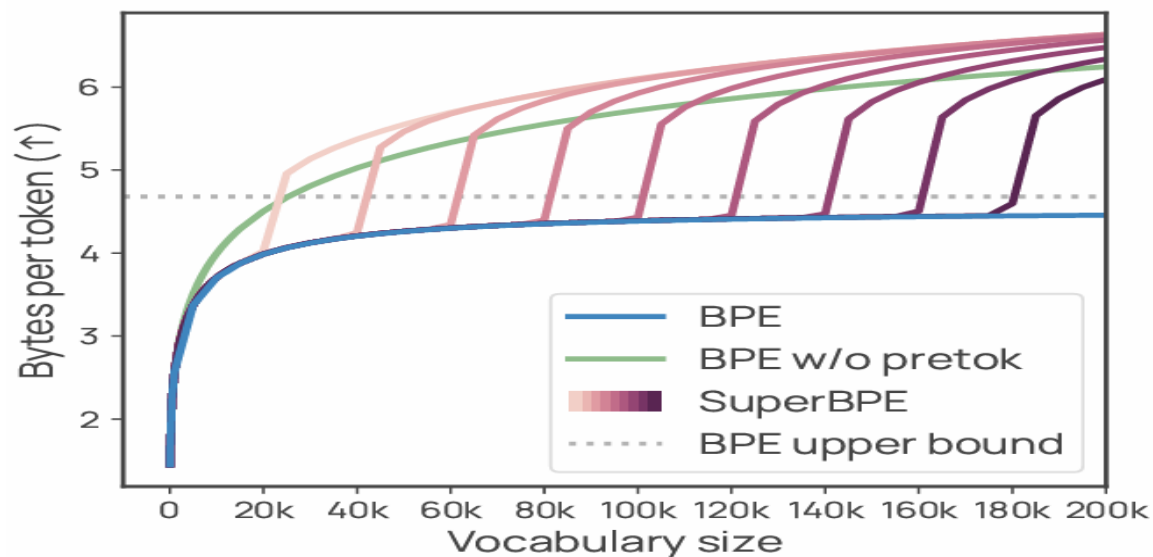


Figure 1: **SuperBPE tokenizers encode text much more efficiently than BPE, and this advantage grows with larger vocabulary size.** Encoding efficiency (y -axis) is measured

출처 : <https://arxiv.org/pdf/2503.13423>

토큰 가격 (Token Price)



Model	Input Price for 1000 tokens (prompt)	Output Price for 1000 tokens (completion)
Ada	\$0.0004	\$0.0016
Babbage	\$0.0005	\$0.0024
Curie	\$0.0020	\$0.0120
DaVinci	\$0.0200	\$0.1200
ChatGPT	\$0.0020	\$0.0020
Chat 4K context	\$0.0015	\$0.002
GPT-4 8k context	\$0.03	\$0.06
Chat 16K context	\$0.003	\$0.004
GPT-4 32k context	\$0.06	\$0.12

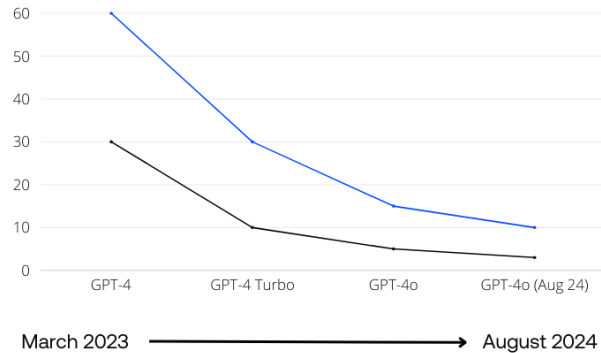
출처 : [What Is the ChatGPT Token Limit and Can You Exceed It? \(makeuseof.com\)](https://makeuseof.com/what-is-the-chatgpt-token-limit-and-can-you-exceed-it/)

토큰 가격 (Token Price) 하락



OpenAI GPT-4 price drops

\$ price per 1 Million tokens ● Output ● Input



Nebuly^{AI}

Model	\$/ 1 million output tokens	\$/ 1 million input tokens
GPT-4 (released March 2023)	60	30
GPT-4 Turbo (released November 2023)	30	10
GPT-4o (released May 2024)	15	5
GPT-4o (update August 2024)	10	3
GPT-4o Mini (released July 2024)	0.6	0.15

Nebuly^{AI}

300쪽짜리 책도 몇초만에 요약 ... 자비 없이 치고나가는 오픈AI

이덕주 기자

입력 : 2023-11-07 17:29:32

수정 : 2023-11-07 23:17:08

오픈AI는 챗GPT를 공개한 지 약 1년이 된 6일(현지시간) '오픈AI 데브데이(DevDay)'에서 후발 주자와 격차를 더욱 벌리기 위한 전략을 여실히 드러냈다.

전 세계에서 개발자 수백 명이 집결한 가운데 가장 큰 주목을 받은 것은 '맞춤형 챗GPT'였다. 그동안 챗GPT와 같은 AI 챗봇을 개인 맞춤형으로 개발하려는 시도는 많았지만 코딩을 모르는 개인에게는 높은 장벽이었다. 오픈AI는 누구든 챗봇을 학습시켜 자신만의 GPT를 구축할 수 있다고 설명했다. 샘 올트먼 오픈AI 최고경영자(CEO)는 'GPT빌더'라는 기능을 통해 스타트업에 조언하는 GPT를 만드는 과정을 시연했다. GPT 프로필 사진을 AI를 통해 생성하고, 그의 원고를 업로드한 후 "스타트업 창업자가 채용할 때 고려할 세 가지는 무엇인가"라고 질문하자 그의 원고에서 언급된 세 가지 항목으로 AI가 답을 했다. AI를 교육시키는 데 걸린 시간은 45초에 불과했다.

기존 챗GPT도 이날 새로운 모델 'GPT-4 터보'로 업그레이드했다. GPT-4 터보는 최대 300페이지까지 입력이 가능하다. 이미지를 인식하는 비전 능력, 이미지를 그리는 달리(DALL·E)-3 능력, 텍스트를 음성으로 바꾸는 TTS 기능을 기본으로 탑재하고 있다. TTS에서는 총 6개 목소리를 제공한다.

한편 이날 행사에는 사티아 나델라 마이크로소프트(MS) CEO가 직접 참석해 오픈AI와의 파트너십이 든든함을 과시했다. MS는 오픈AI 주요 투자자인데, 오픈AI의 AI 서비스를 자신들의 클라우드 인프라스트럭처를 통해 고객들에게 제공하고 있다. MS는 오픈AI의 AI로 만든 서비스를 365, 윈도 등에 코파일럿이라는 이름으로 서비스하고 있다.

나델라 CEO는 "나는 인프라 비즈니스를 30년 넘게 해왔는데 오픈AI와의 파트너십으로 MS 애저도 많은 변화를 했다"면서 "두 회사의 협력은 개발자들이 새로운 것을 만드는 데 큰 도움을 줄 것"이라고 설명했다.

더 똑똑해진 GPT-4 터보
올해 4월까지 정보 수록하고
이미지 인식·생성 기능 탑재

AI 생태계 구축 어떻게
맞춤형 GPT 만들어 올리면
애플처럼 앱스토어서 AI 거래

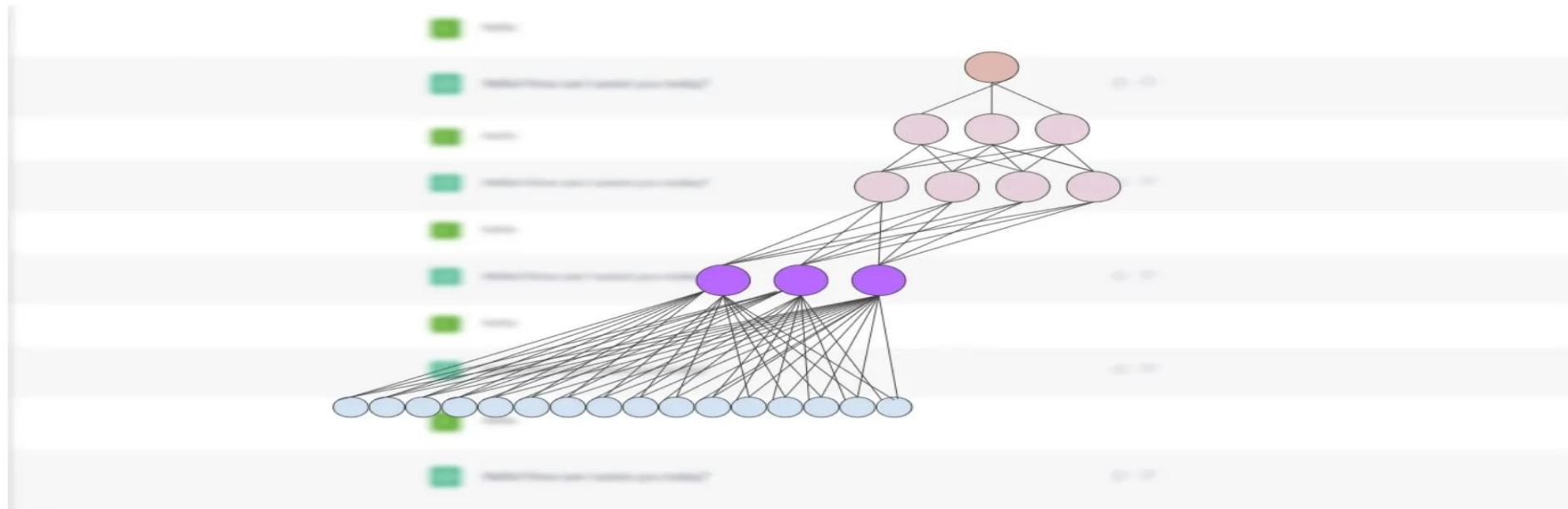
구글·메타 주춤할때 질주
스타트업 진입장벽 높아져
'사다리 건너차기' 평가도

ChatGPT의 숨겨진 무기 – 임베딩(Embedding)



Embeddings: ChatGPT's Secret Weapon

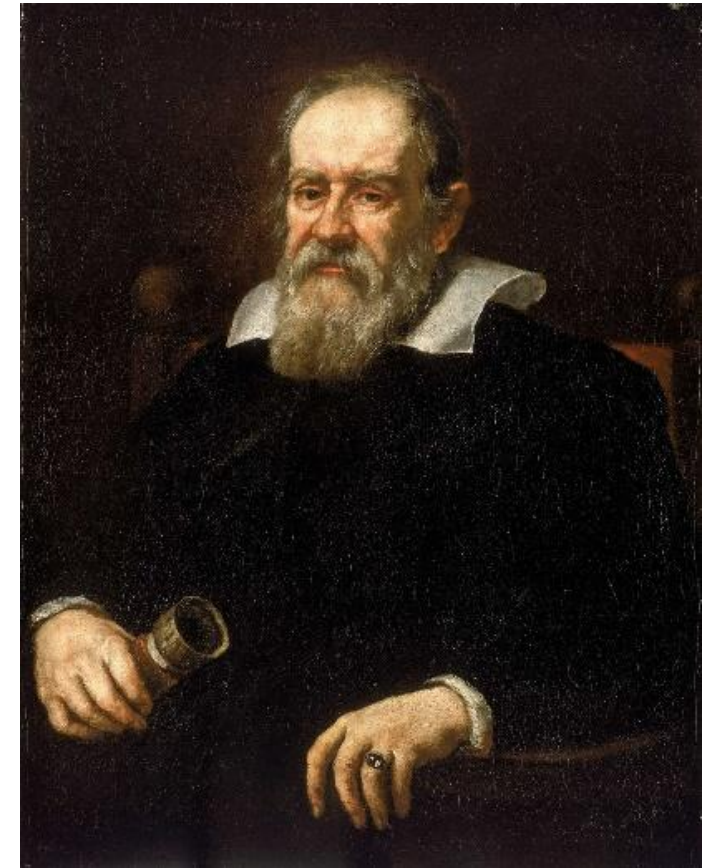
Embeddings, and how they help ChatGPT predict the next word



(image by author)

- 컴퓨터가 다루는(다룰 수 있는) 것은 숫자
- 인공지능은 숫자를 예측한다.

과학적 접근의 기초 → 수치화 (벡터화)

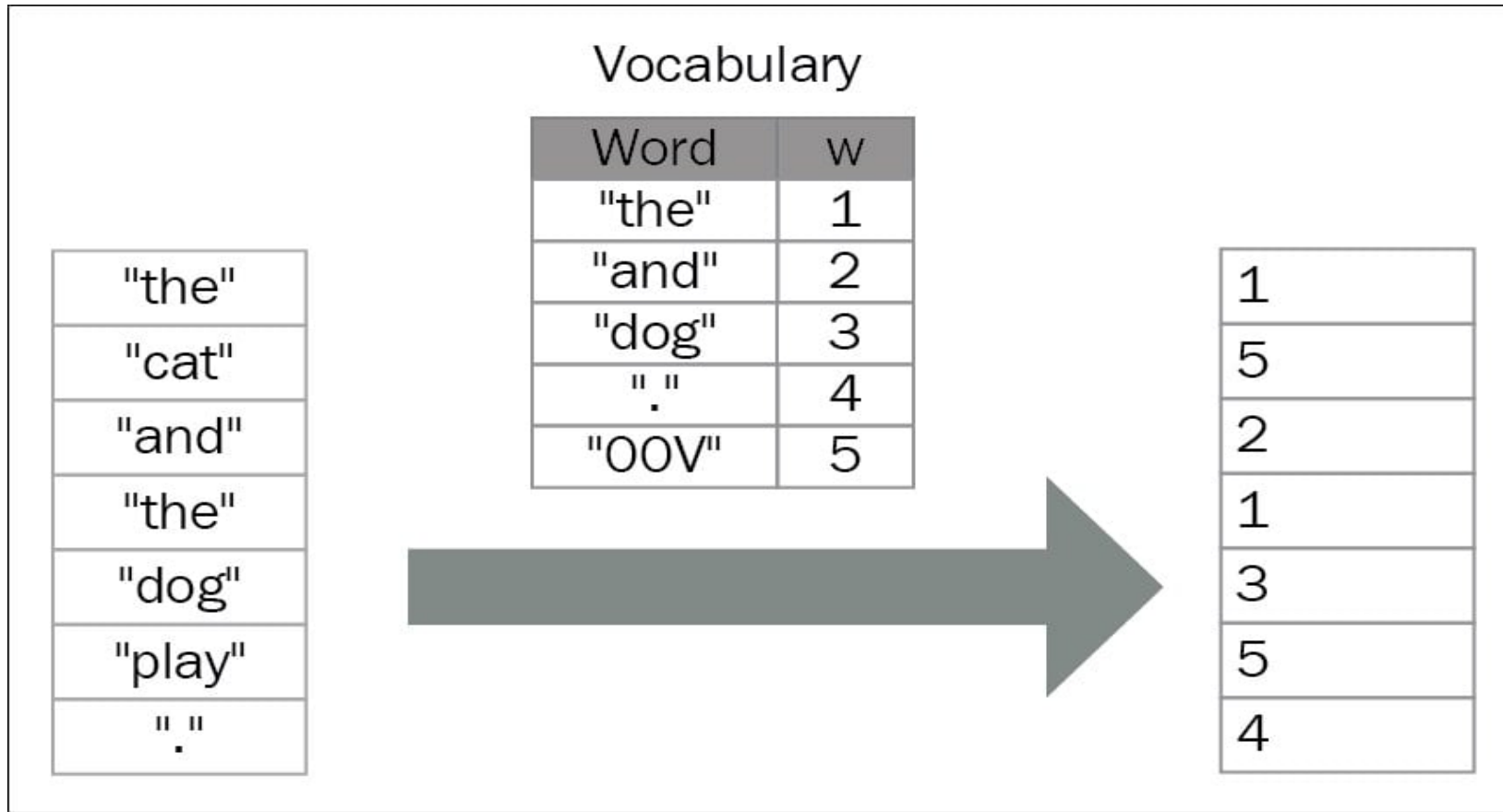


셀 수 있는 모든 것을 세고, 잴 수 있는 모든 것을 재어라.

그리고 잴 수 없는 것은 잴 수 있게 만들어라

– 갈릴레오

텍스트의 수치화 : 벡터화 – 임의의 수치에 대응



단어	코드
사과	1
치킨	2
바나나	3
양배추	4

출처 : <https://subscription.packtpub.com/book/data/9781786465825/3/ch03lvl1sec32/encoding-and-embedding>

- 언어의 의미를 담지 못한 수치화

Diagram illustrating One-Hot encoding for words:

Arrows indicate the mapping of words to specific indices in the vector:

- Rome points to index 1
- Paris points to index 2
- word V points to index V

One-Hot encoding vectors:

$$\begin{aligned} \text{Rome} &= [1, 0, 0, 0, 0, 0, \dots, 0] \\ \text{Paris} &= [0, 1, 0, 0, 0, 0, \dots, 0] \\ \text{Italy} &= [0, 0, 1, 0, 0, 0, \dots, 0] \\ \text{France} &= [0, 0, 0, 1, 0, 0, \dots, 0] \end{aligned}$$

출처 : <https://medium.com/intelligentmachines/word-embedding-and-one-hot-encoding-ad17b4bbe111>

- 단어 표현에 큰 벡터가 필요 (One-Hot encoding)
- 언어의 의미를 담지 못한 수치화

텍스트의 수치화 : 벡터화 - 지정된 특성을 담은 수치에 대응



	Man (5391)	Woman (9853)	King (4914)	Queen (7157)	Apple (456)	Orange (6257)
Gender	-1	1	-0.95	0.97	0.00	0.01
Royal	0.01	0.02	<u>0.93</u>	<u>0.95</u>	-0.01	0.00
Age	0.03	0.02	0.7	0.69	0.03	-0.02
Food	0.04	0.01	0.02	0.01	0.95	0.97
⋮						

출처 : <https://medium.com/intelligentmachines/word-embedding-and-one-hot-encoding-ad17b4bbe111>

- 단어의 특성을 결정하기 어려움
- 선택한 특성 수에 따라 벡터의 차원이 결정됨

텍스트의 수치화 : 훈련에 의한 임베딩 - 딥러닝 방식



- 빈도 기반 분석의 문제점

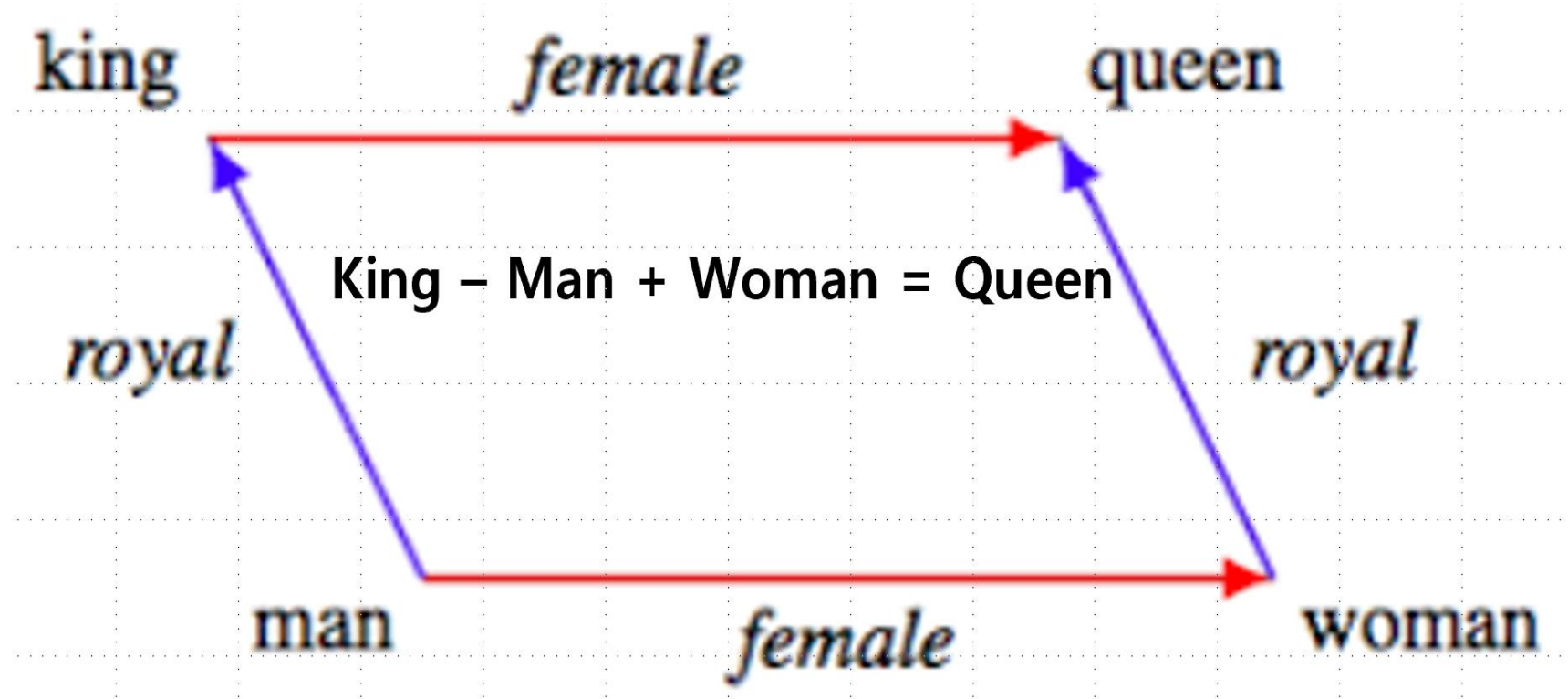
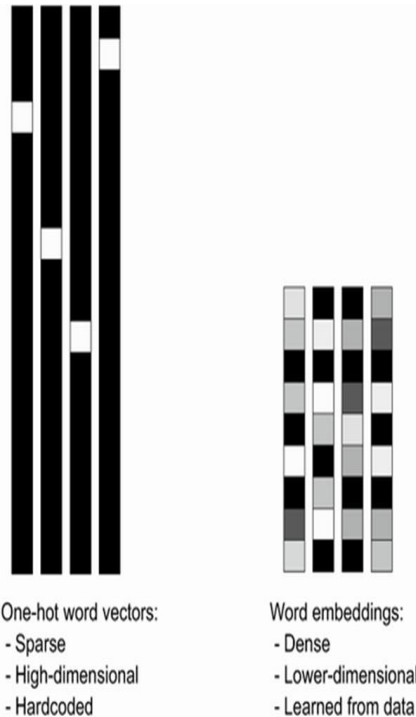
- 단어 표현에 큰 벡터가 필요 (One-Hot encoding)
- 언어의 의미를 담지 못한 수치화

- 언어(단어)의 의미를 수치에 담을 수 있을까?

- 문장은 의미를 갖는다.
- 같은 문장 내의 단어는 하나의 의미를 전달한다.
- 문장 내의 단어를 가까운 위치에 존재하게 만들자.

→ Word2Vec





- 언어의 벡터화

- ✓ Count 기반, Tf-Idf → 통계적 방식
- ✓ 훈련에 의한 생성 → 딥러닝 (특성 추출 : Feature Extraction)

```
model.most_similar(positive=['king','woman'], negative=['man'],topn=10)
Out[12]:
[('queen', 0.7118192911148071),
 ('monarch', 0.6189674139022827),
 ('princess', 0.5902431607246399),
 ('crown_prince', 0.5499460697174072),
 ('prince', 0.5377321243286133),
 ('kings', 0.5236844420433044),
 ('Queen_Consort', 0.5235945582389832),
 ('queens', 0.5181134343147278),
 ('sultan', 0.5098593235015869),
 ('monarchy', 0.5087411999702454)]
```

훈련된 문장에서 거리를 배운다
→ 영역 별 거리도 만든다

임베딩(Embedding) 예 : Word2Vec



King - Man + Woman

$$(2, 5) - (1, 3) + (1, 4) = (2, 6)$$

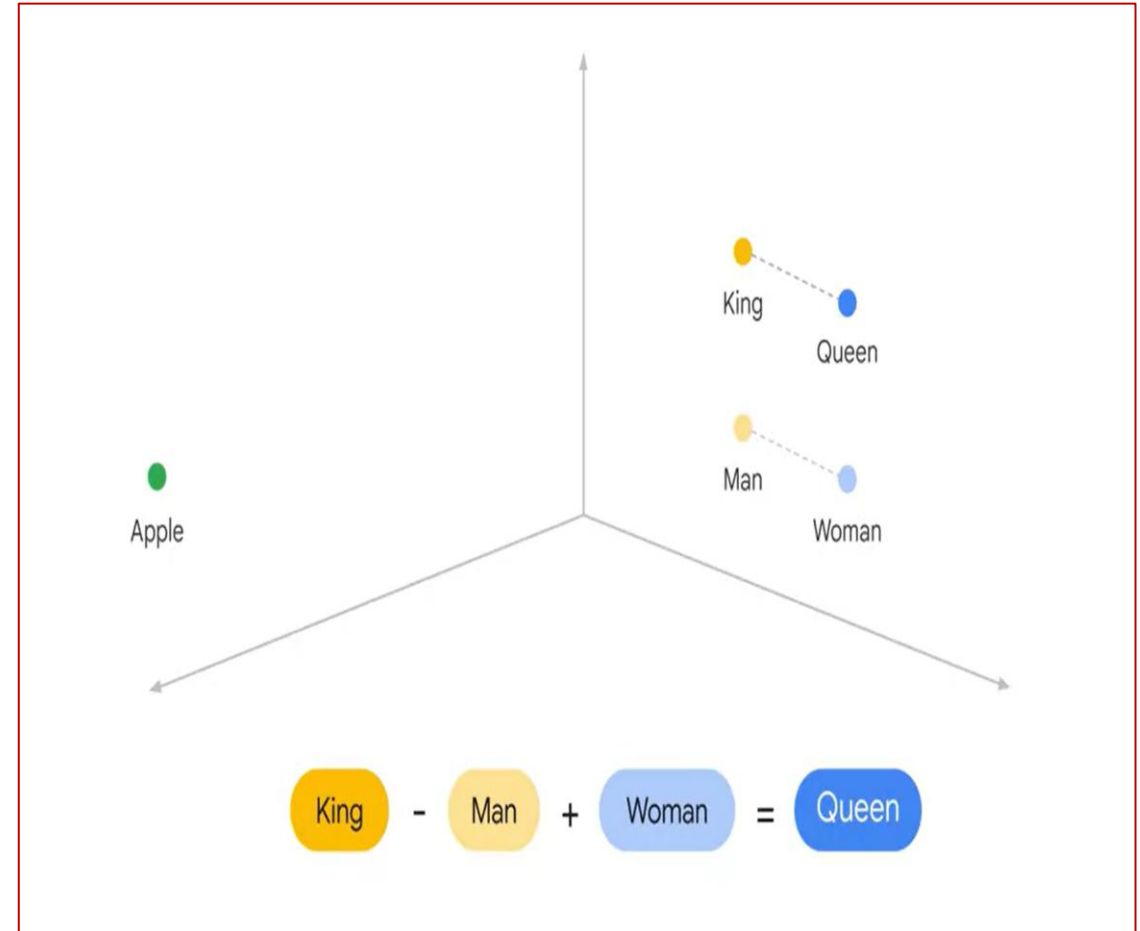
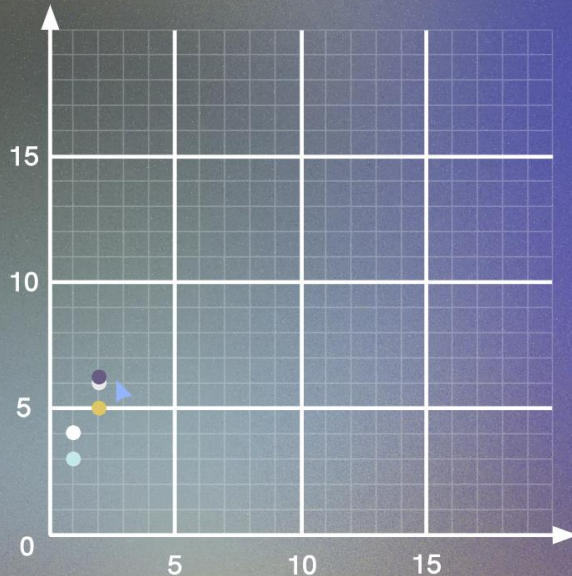
King (2, 5)

Man (1, 3)

Woman (1, 4)

(2, 6)

Queen (2, 6.2)



텍스트의 수치화 : 벡터화

중심 단어 주변 단어

The fat cat sat on the mat

The fat cat sat on the mat

The fat cat sat on the mat

The fat cat sat on the mat

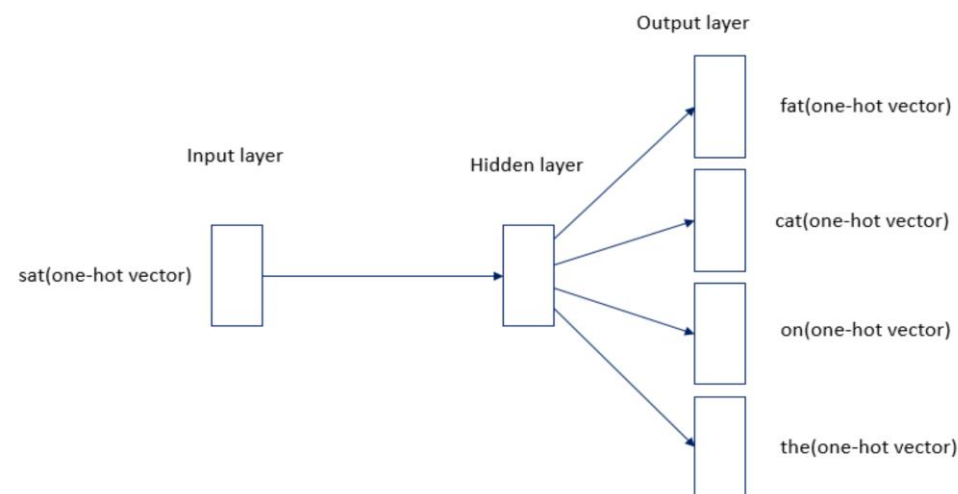
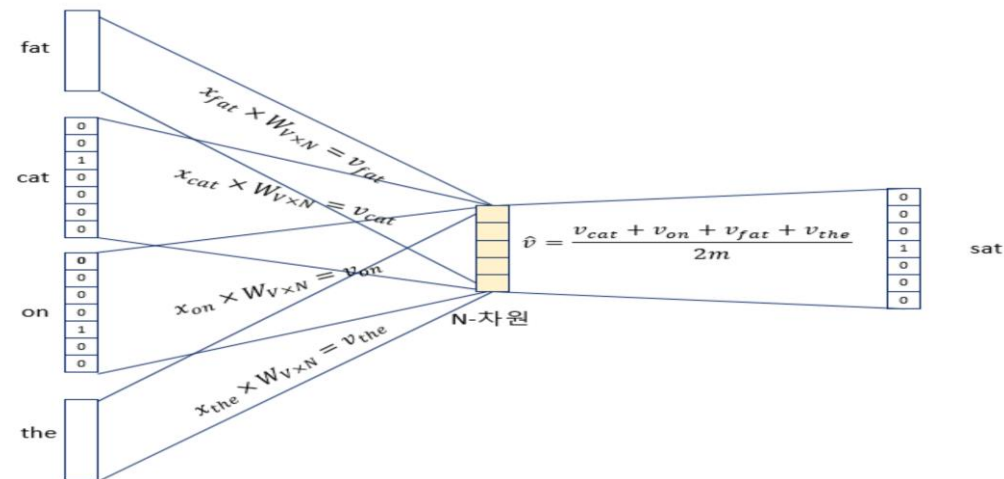
The fat cat sat on the mat

The fat cat sat on the mat

The fat cat sat on the mat

중심 단어	주변 단어
[1, 0, 0, 0, 0, 0, 0]	[0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0]
[0, 1, 0, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0]
[0, 0, 1, 0, 0, 0, 0]	[1, 0, 0, 0, 0, 0, 0], [0, 1, 0, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0]
[0, 0, 0, 1, 0, 0, 0]	[0, 1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0]
[0, 0, 0, 0, 1, 0, 0]	[0, 0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 0, 1, 0]	[0, 0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 0, 0, 1]	[0, 0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1, 0]

(그림3) 출처 : <https://wikidocs.net/22660>



Word2Vec 훈련시키기



```
from gensim.models import Word2Vec
import matplotlib.pyplot as plt
```

```
# 단어와 2차원 x축의 값, y축의 값을 입력받아 2차원 그래프를 그린다
def plot_2d_graph(vocabs, xs, ys):
    plt.figure(figsize=(8,6))
    plt.scatter(xs, ys, marker='o')
    for i, v in enumerate(vocabs):
        plt.annotate(v, xy=(xs[i], ys[i]))
```

```
sentences = [
    ['this', 'is', 'a', 'good', 'product'],
    ['it', 'is', 'a', 'excellent', 'product'],
    ['it', 'is', 'a', 'bad', 'product'],
    ['that', 'is', 'the', 'worst', 'product']
]
```

```
# 문장을 이용하여 단어와 벡터를 생성한다.
```

```
model = Word2Vec(sentences, size=300, window=3, min_count=1, workers=1)
```

```
# 단어벡터를 구한다.
```

```
word_vectors = model.wv
```

```
vocabs = word_vectors.vocab.keys()
```

```
word_vectors_list = [word_vectors[v] for v in vocabs]
```

```
# 단어간 유사도를 확인하다
```

```
print(word_vectors.similarity(w1='it', w2='this'))
```

-0.022223482

Word2Vec 임베딩 테스트



```
test_words = ["espresso"]
out_words = model.most_similar(positive=test_words)
print( test_words[0], " most similar ===== ")
print_list(out_words)
```

```
espresso most similar =====
0 == ('cappuccino', 0.6888187527656555)
1 == ('mocha', 0.6686209440231323)
2 == ('coffee', 0.6616825461387634)
3 == ('latte', 0.6536751985549927)
4 == ('espressos', 0.6438628435134888)
5 == ('macchiato', 0.6428249478340149)
6 == ('brewed_coffee', 0.6234086751937866)
7 == ('iced_coffee', 0.6213865280151367)
8 == ('expresso', 0.6108267307281494)
9 == ('coffees', 0.6038862466812134)
```

```
test_words = ["samsung"]
out_words = model.most_similar(positive=test_words)
print( test_words[0], " most similar ===== ")
print_list(out_words)
```

```
samsung most similar =====
0 == ('htc', 0.7326198816299438)
1 == ('nokia', 0.7076619267463684)
2 == ('sony', 0.6818498373031616)
3 == ('motorola', 0.6689046025276184)
4 == ('ipad', 0.6625324487686157)
5 == ('iphone', 0.6582239866256714)
6 == ('macbook', 0.6297854781150818)
7 == ('verizon', 0.6176369786262512)
8 == ('ipod_touch', 0.6170758008956909)
9 == ('ps3', 0.607496440410614)
```

```
# Public Opinion
country = 'Japan'

pos_feeling = model.similarity(country, 'positive')
neg_feeling = model.similarity(country, 'negative')

print( country, " pos/neg == ", pos_feeling/neg_feeling)
```

```
Japan pos/neg == 1.1084418
```

```
# Public Opinion
country = 'Korea'

pos_feeling = model.similarity(country, 'positive')
neg_feeling = model.similarity(country, 'negative')

print( country, " pos/neg == ", pos_feeling/neg_feeling)
```

```
Korea pos/neg == 0.90629506
```

```
# Public Opinion
country = 'USA'

pos_feeling = model.similarity(country, 'positive')
neg_feeling = model.similarity(country, 'negative')

print( country, " pos/neg == ", pos_feeling/neg_feeling)
```

```
USA pos/neg == 34.609974
```

고전적 언어 모델 vs 딥러닝 언어 모델



승우

You

다음 세 문장, 각각의 유사도를 0.00 ~ 1.00 사이의 값으로 표시해 줘.

1. It is good.
2. It is not good.
3. It's great.

```
python Copy code

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# 주어진 문장
sentences = [
    "It is good.",
    "It is not good.",
    "It's great."
]

# TF-IDF 벡터화
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(sentences)

# 코사인 유사도 계산
cosine_similarities = cosine_similarity(tfidf_matrix, tfidf_matrix)

# 결과 출력
for i in range(len(sentences)):
    for j in range(i+1, len(sentences)):
        print(f"유사도 ({i+1}, {j+1}): {cosine_similarities[i][j]}")
```

결과:

유사도 (1, 2): 0.4307690062525092

유사도 (1, 3): 0.0

유사도 (2, 3): 0.0

승우

You

User

다음 세 문장, 각각의 유사도를 벡터 임베딩 방법을 사용해서 0.00 ~ 1.00 사이의 값으로 표시해 줘.

1. It is good.
2. It is not good.
3. It's great.

```
python Copy code

import tensorflow_hub as hub
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

# Universal Sentence Encoder 모델 로드
embed = hub.load("https://tfhub.dev/google/universal-sentence-encoder/4")

# 주어진 문장
sentences = [
    "It is good.",
    "It is not good.",
    "It's great."
]

# 문장을 벡터로 임베딩
sentence_embeddings = embed(sentences)

# 코사인 유사도 계산
cosine_similarities = cosine_similarity(sentence_embeddings)

# 결과 출력
for i in range(len(sentences)):
    for j in range(i+1, len(sentences)):
        print(f"유사도 ({i+1}, {j+1}): {cosine_similarities[i][j]}")
```

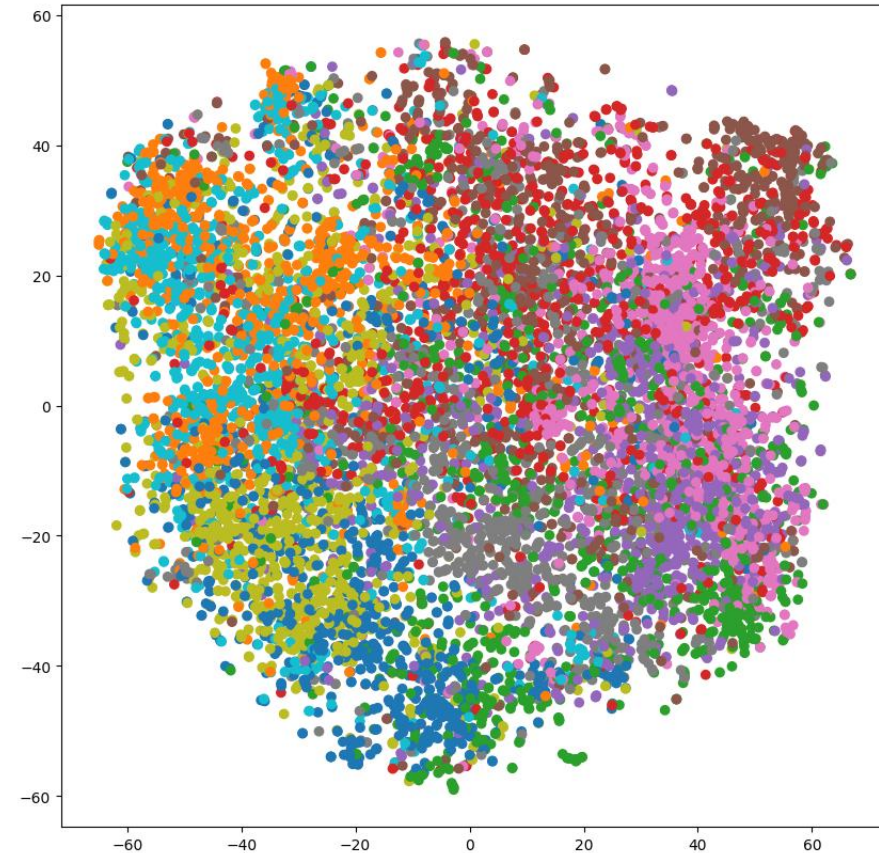
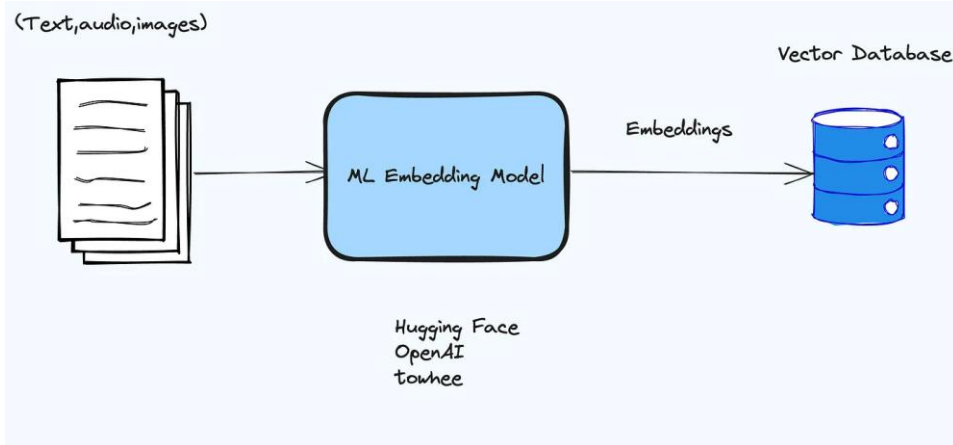
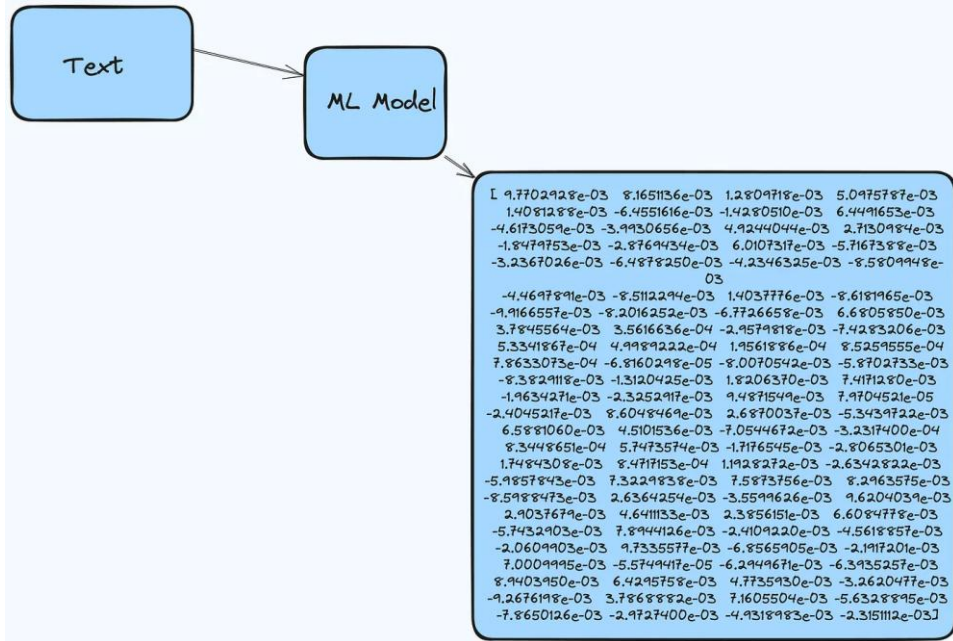
결과:

유사도 (1, 2): 0.4776627128124237

유사도 (1, 3): 0.5661327838897705

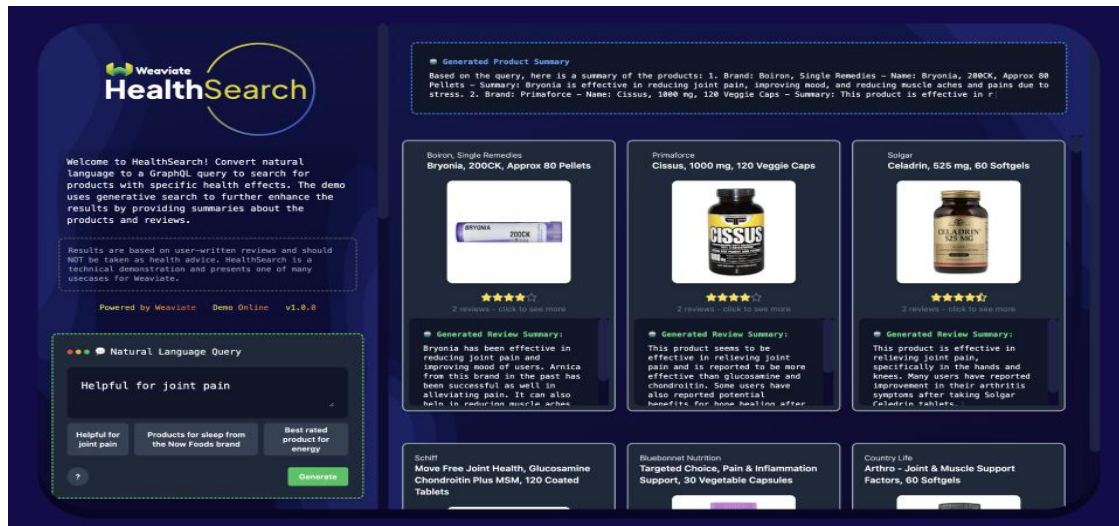
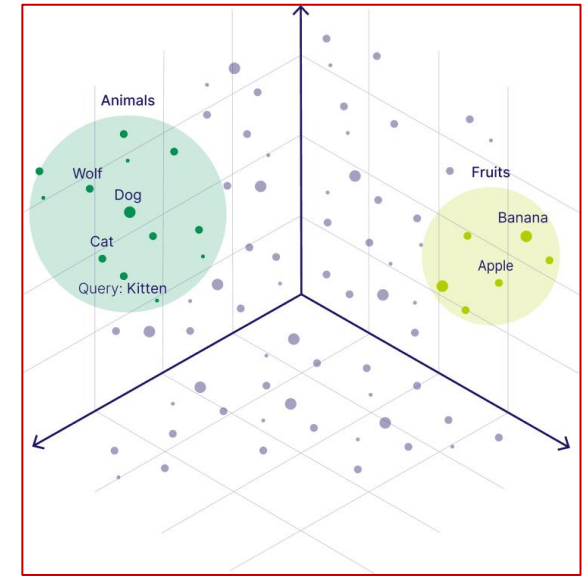
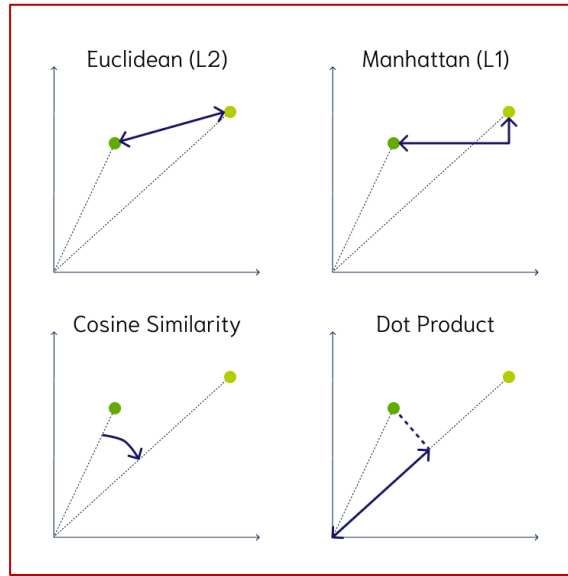
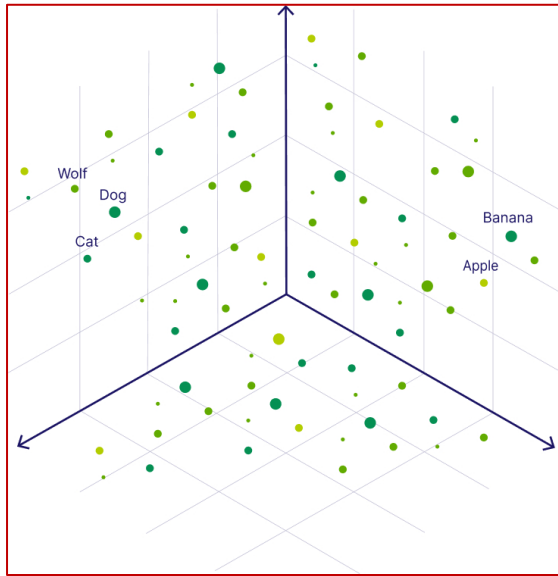
유사도 (2, 3): 0.2884486310482025

임베딩 : 수치화 (벡터화)



벡터 공간에서는
의미론적 유사도 비교가 가능하다.

임베딩 : (비정형 데이터의) 의미 비교 가능



Traditional Search

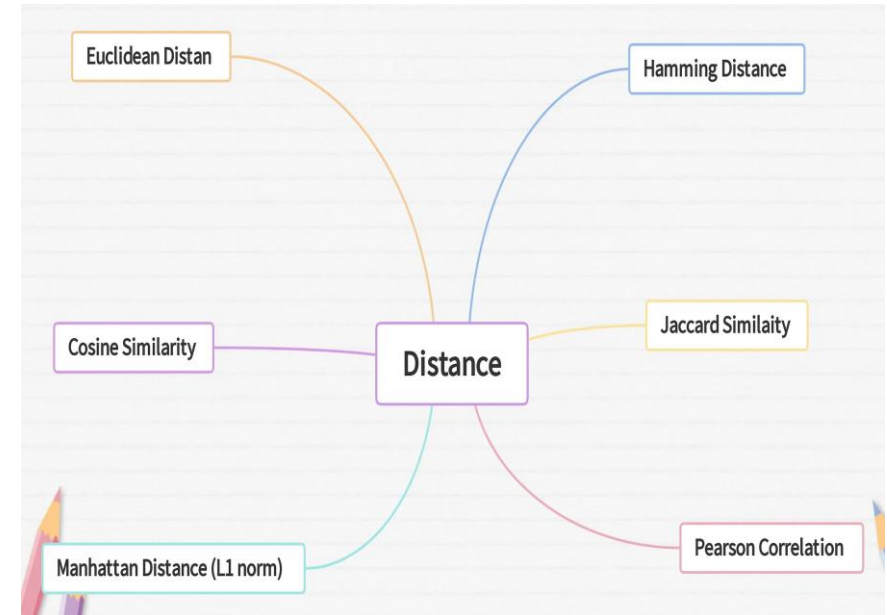
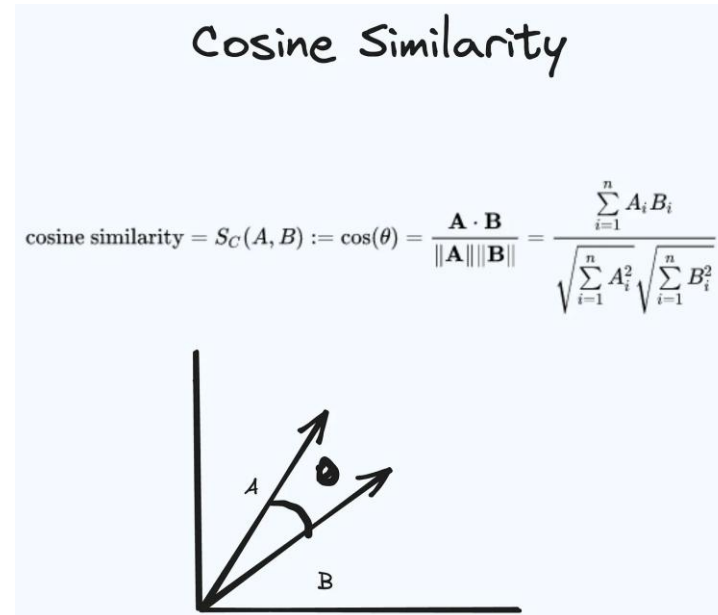
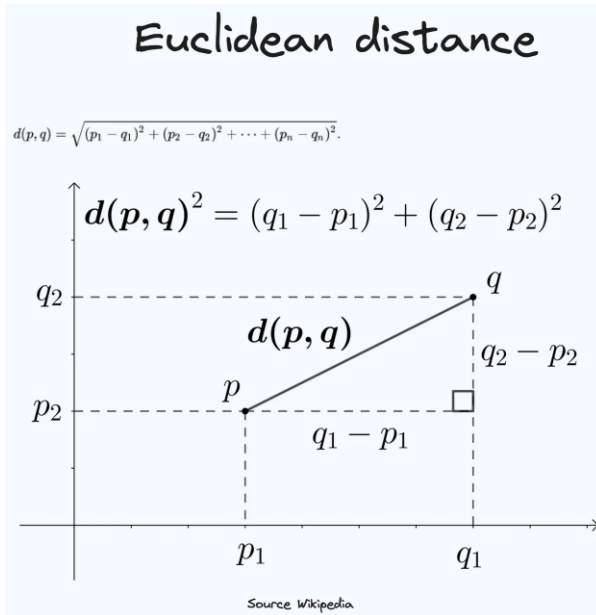
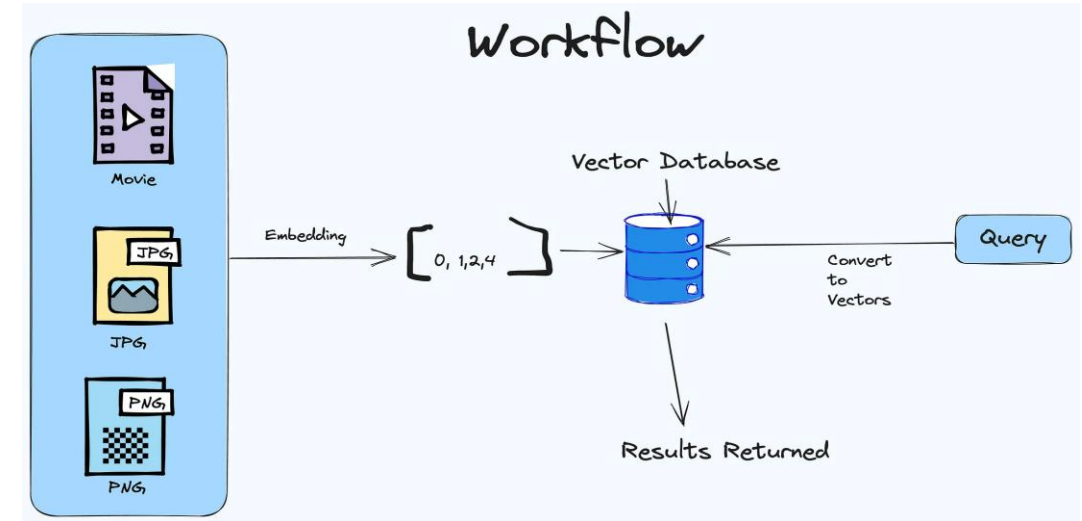
```
SELECT question
FROM JeopardyQuestions
WHERE (question LIKE '%dog%' OR
question LIKE '%cat%' OR
question LIKE '%wolf%' OR
(... and so on))
```

VS

Semantic Search

```
{
  Get {
    JeopardyQuestions {
      nearText: { concepts: ["animals"] }
    } { question }
  }
}
```

임베딩 : 유사도 비교 가능 (Q: 무엇의 유사도?, 스타일? 콘텐츠?)



임베딩 : 의미 차이 vs 언어 차이



다른 언어 문서가 임베딩된 벡터 데이터베이스에서 영어로 질문을 할 경우, 검색 정확도는?

```
7 corpus = [  
8     '한 남자가 음식을 먹는다.',  
9     '한 남자가 빵 한 조각을 먹는다.',  
10    '그 여자가 아이를 돌본다.',  
11    '한 남자가 말을 탄다.',  
12    '한 여자가 바이올린을 연주한다.',  
13    '두 남자가 수레를 숲 속으로 밀었다.',  
14    '한 남자가 담으로 싸인 땅에서 백마를 타고 있다.',  
15    '원숭이 한 마리가 드럼을 연주한다.',  
16    '치타 한 마리가 먹이 뒤에서 달리고 있다.',  
17    'A man is eating food.',  
18    'A man is eating a piece of bread.',  
19    'The woman is looking after the child.',  
20    'A man is riding a horse.',  
21    'A woman is playing the violin.',  
22    'Two men are pushing a cart into the forest.',  
23    'A man is riding a white horse in a fenced-in area.',  
24    'A monkey is playing a drum.',  
25    'A cheetah is running behind its prey.',  
26 ]
```

```
32 # Query sentences:  
33 queries = [  
34     '한 남자가 파스타를 먹는다.',  
35     '고릴라 의상을 입은 누군가가 드럼을 연주하고 있다.',  
36     '치타가 들판을 가로 질러 먹이를 쫓는다.',  
37     'A man eats pasta.',  
38     'Someone in a gorilla suit plays drums.',  
39     'A cheetah chases its prey across a field.',  
40 ]
```

```
embedder = SentenceTransformer("jhgan/ko-sbert-sts")
```

```
11 # openai.api_key = OPENAI_API_KEY  
12 client = OpenAI(api_key = OPENAI_API_KEY )
```


임베딩 : 의미 차이 vs 언어 차이



한국어 임베딩은 한글 질문에, 영어 임베딩은 영어 질문에 보다 정확한 검색 ???

Query: 한 남자가 파스타를 먹는다.

Top 5 most similar sentences in corpus:

한 남자가 음식을 먹는다. (Score: 0.6154)
한 남자가 빵 한 조각을 먹는다. (Score: 0.5451)
A man is eating food. (Score: 0.2438)
A man is eating a piece of bread. (Score: 0.1979)
한 여자가 바이올린을 연주한다. (Score: 0.0980)

Query: 한 남자가 파스타를 먹는다.

Top 5 most similar sentences in corpus:

한 남자가 음식을 먹는다. (Score: 0.7863)
한 남자가 빵 한 조각을 먹는다. (Score: 0.7145)
한 남자가 말을 탄다. (Score: 0.5641)
한 남자가 담으로 싸인 땅에서 백마를 타고 있다. (Score: 0.4793)
두 남자가 수레를 숲 속으로 밀었다. (Score: 0.4099)

Query: A man eats pasta.

Top 5 most similar sentences in corpus:

A man is eating food. (Score: 0.7292)
A man is eating a piece of bread. (Score: 0.5975)
A man is riding a horse. (Score: 0.4315)
A monkey is playing a drum. (Score: 0.3794)
한 남자가 음식을 먹는다. (Score: 0.3765)

Query: A man eats pasta.

Top 5 most similar sentences in corpus:

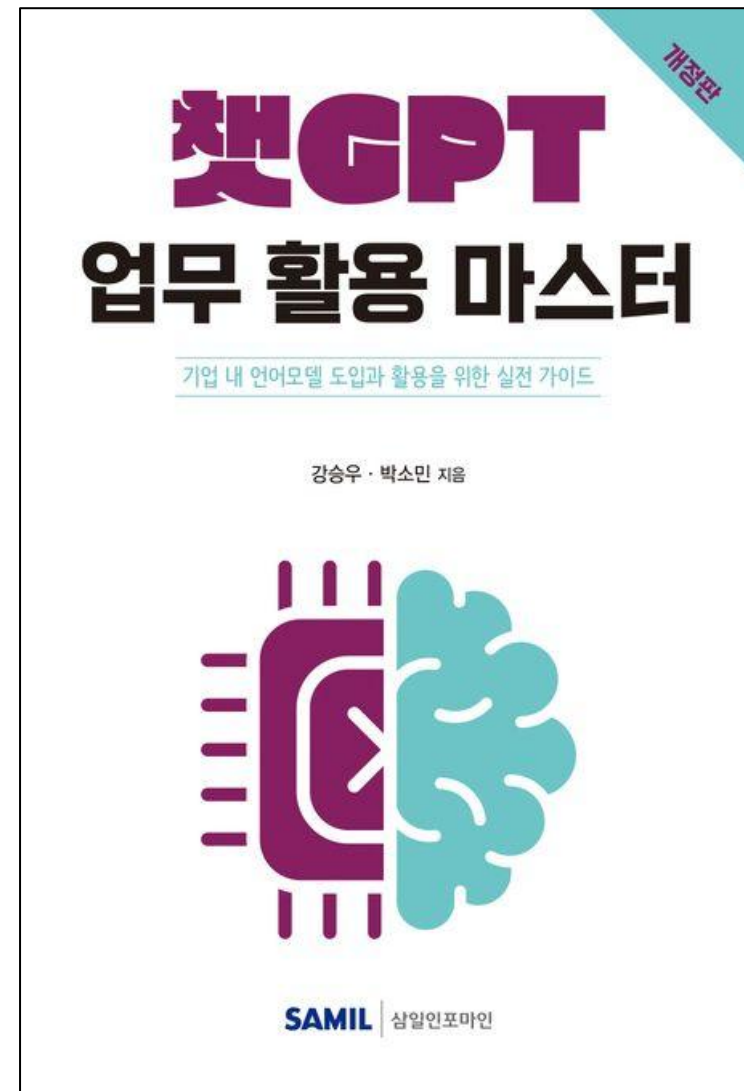
A man is eating food. (Score: 0.6052)
A man is eating a piece of bread. (Score: 0.4747)
한 남자가 음식을 먹는다. (Score: 0.4281)
한 남자가 빵 한 조각을 먹는다. (Score: 0.4038)
A man is riding a horse. (Score: 0.2266)

```
embedder = SentenceTransformer("jhgan/ko-sbert-sts")
```

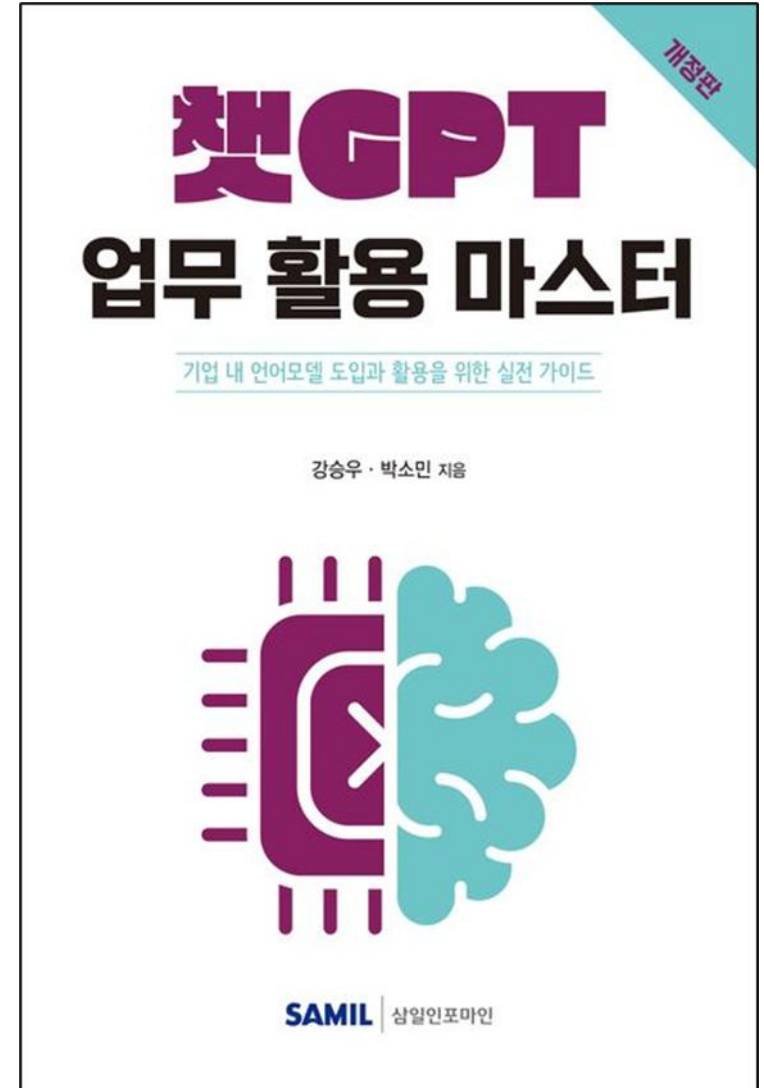
```
11 # openai.api_key = OPENAI_API_KEY  
12 client = OpenAI(api_key = OPENAI_API_KEY )
```

요약

- 자연어 처리(NLP) = 자연어 이해(NLU) + 자연어 생성(NLG)
- 임베딩(Embeddings) : 의미를 담은 숫자
- 토큰(Token) : 언어 AI 모델의 최소 처리 단위
- 다양한 생성형 (무료) 언어 모델 사이트
 - ✓ ChatGPT : <https://chat.openai.com/>
 - ✓ 뽀튼 : <https://wrtn.ai/>
 - ✓ 클로바 X : <https://clova-x.naver.com>
 - ✓ 클로드3 : <https://claude.ai/chat/>
 - ✓ MS Copilot(bing) : <https://www.bing.com/>
 - ✓ 구글 Gemini(Bard) : <https://gemini.google.com/>
 - ✓ 퍼플렉시티 (Perplexity) : <https://www.perplexity.ai/>



- Bing Create : <https://www.bing.com/Create>
- Suno : <https://www.suno.ai/>
- Runway : <https://runwayml.com/>
- Gamma : <https://gamma.app/>
- Elicit : <https://elicit.com/>
-



감사합니다.